IMEN891K / IDSC501 / AXGS-501 · DATA ENGINEERING · FALL 2026

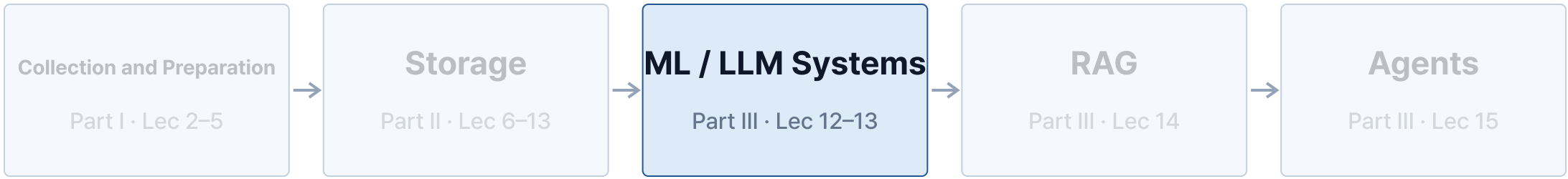
# 14.

# ML Systems

Sangwoo Mo · POSTECH

Industrial & Management Engineering

# Where We Are




Part III connects stored data to models, retrieval, and actions

# Outline: ML Systems

01

## From Data to Predictions

prepare inputs and use a trained model

02

## Building and Releasing Models

check models before putting them into use

03

## Monitoring Models in Use

notice changes and update the model

# 01 · From Data to Predictions

01

**From Data to Predictions**

prepare inputs and use a trained model

02

**Building and Releasing Models**

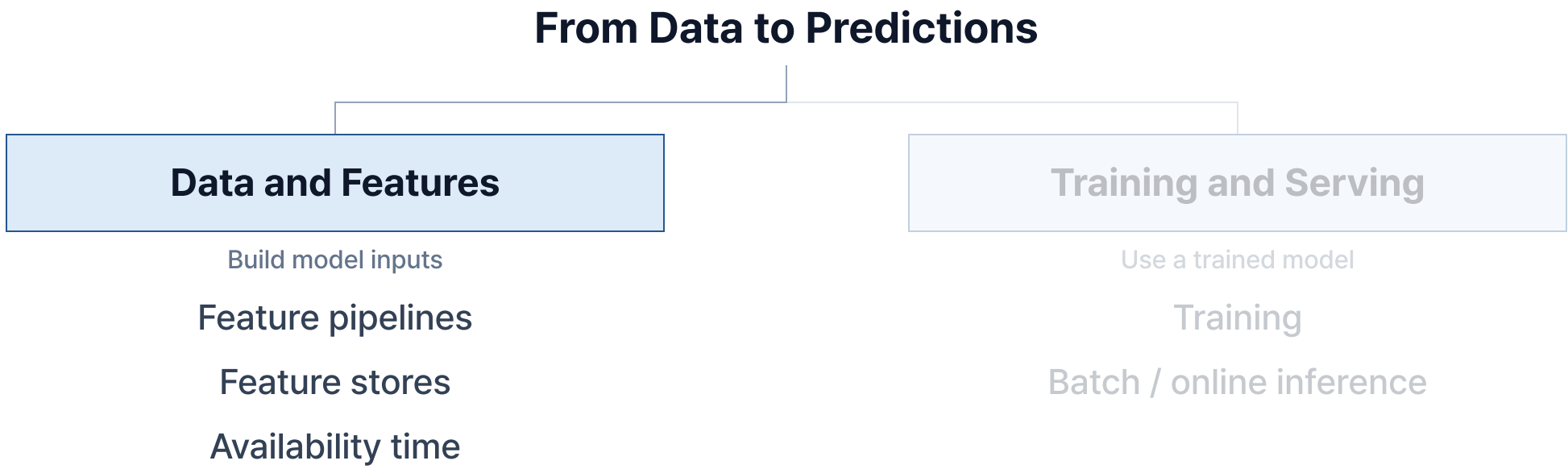
check models before putting them into use

03

**Monitoring Models in Use**

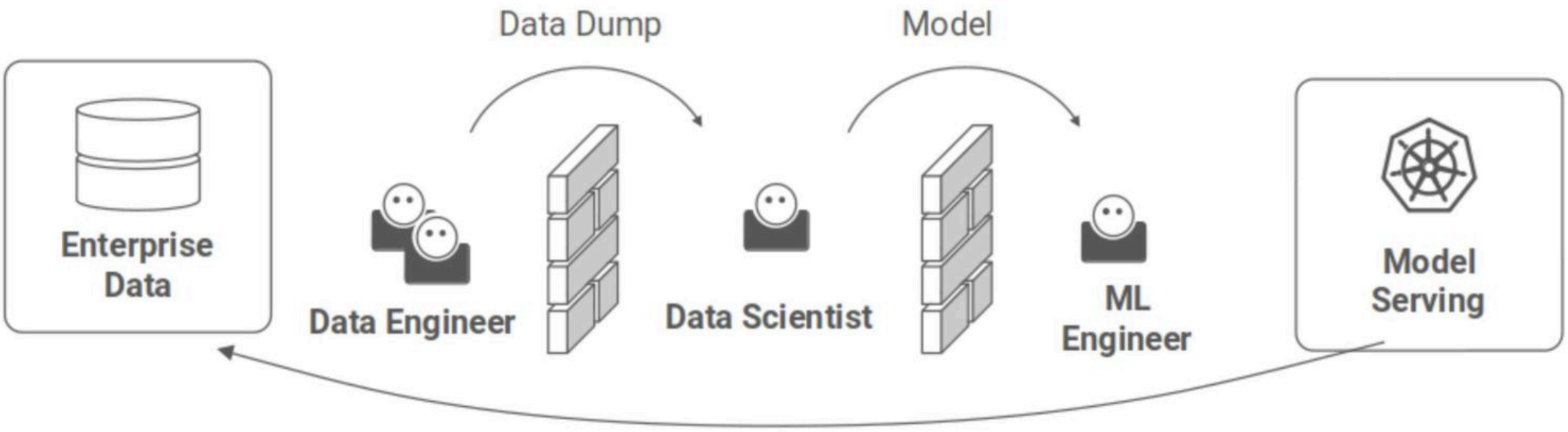
notice changes and update the model

# § Data and Features



# Data Scientist and ML Engineer

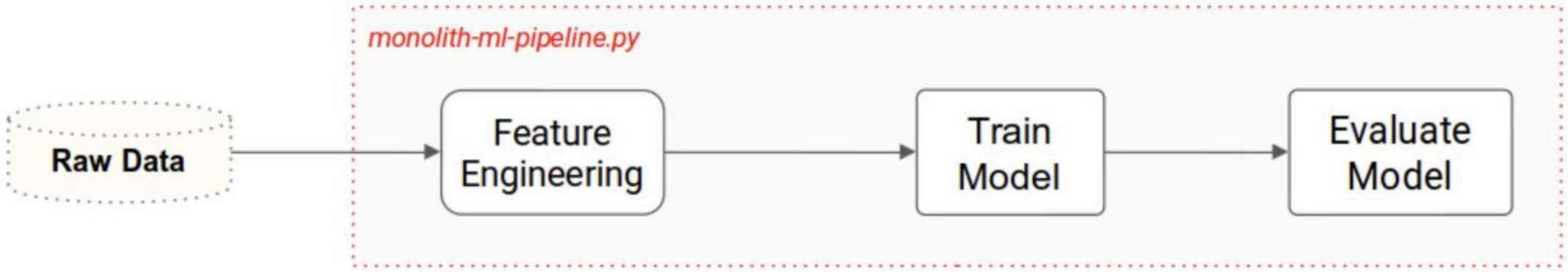
A model needs a reliable path from source data to predictions




Roles span data preparation, training, and serving

# ML Pipeline

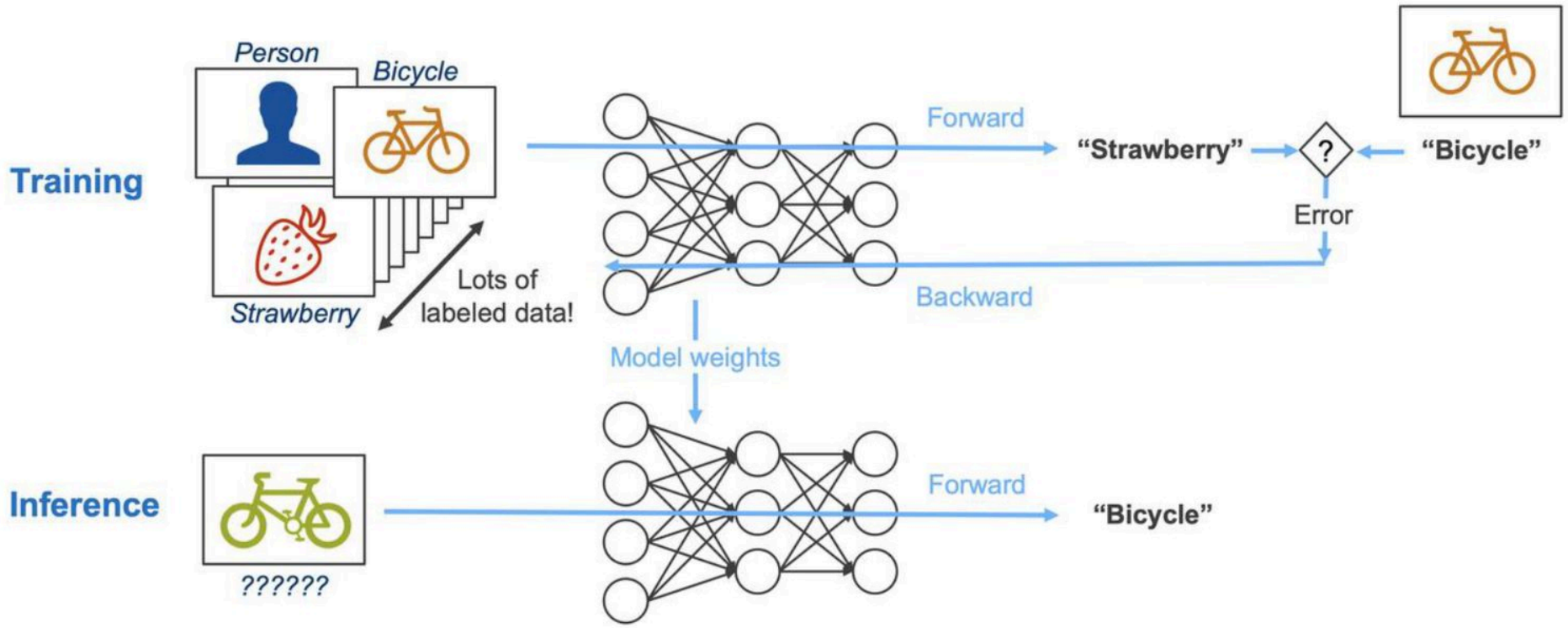
Training transforms prepared examples into an evaluated model




Features, training, and evaluation form one pipeline

# Training and Inference

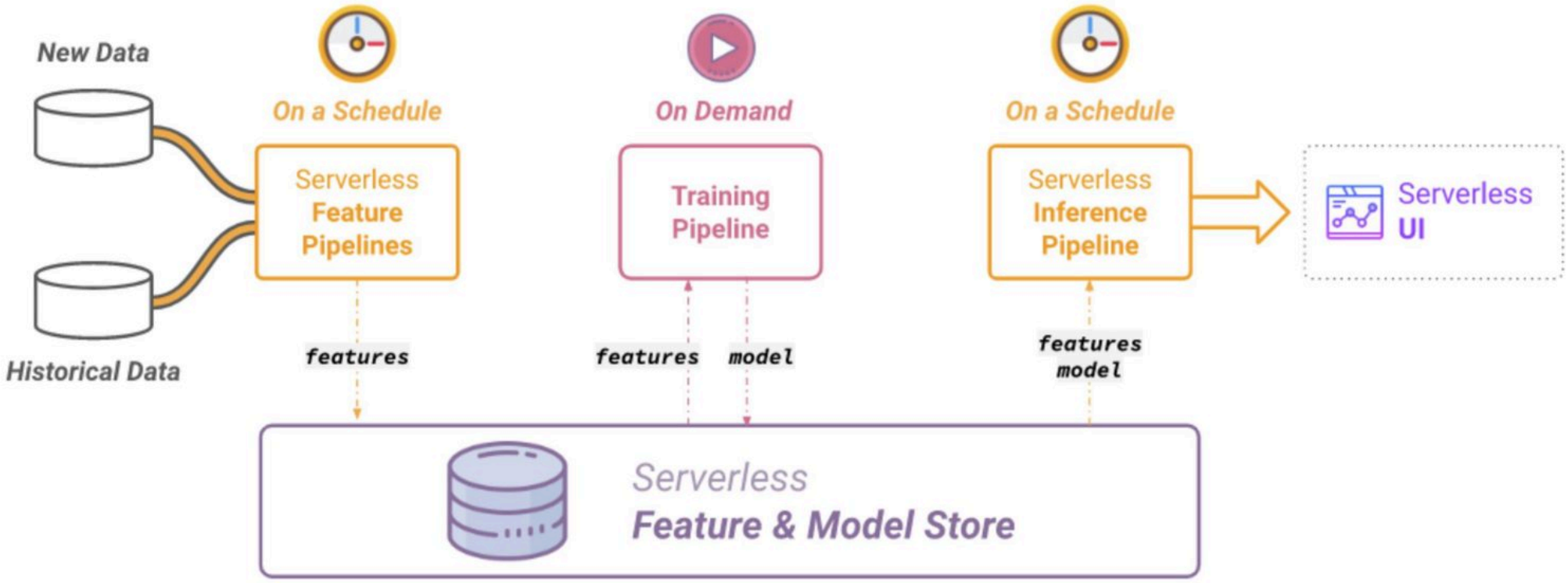
Inference applies the learned model to new inputs




Training uses labels; inference receives new inputs

# ML Pipeline at Scale

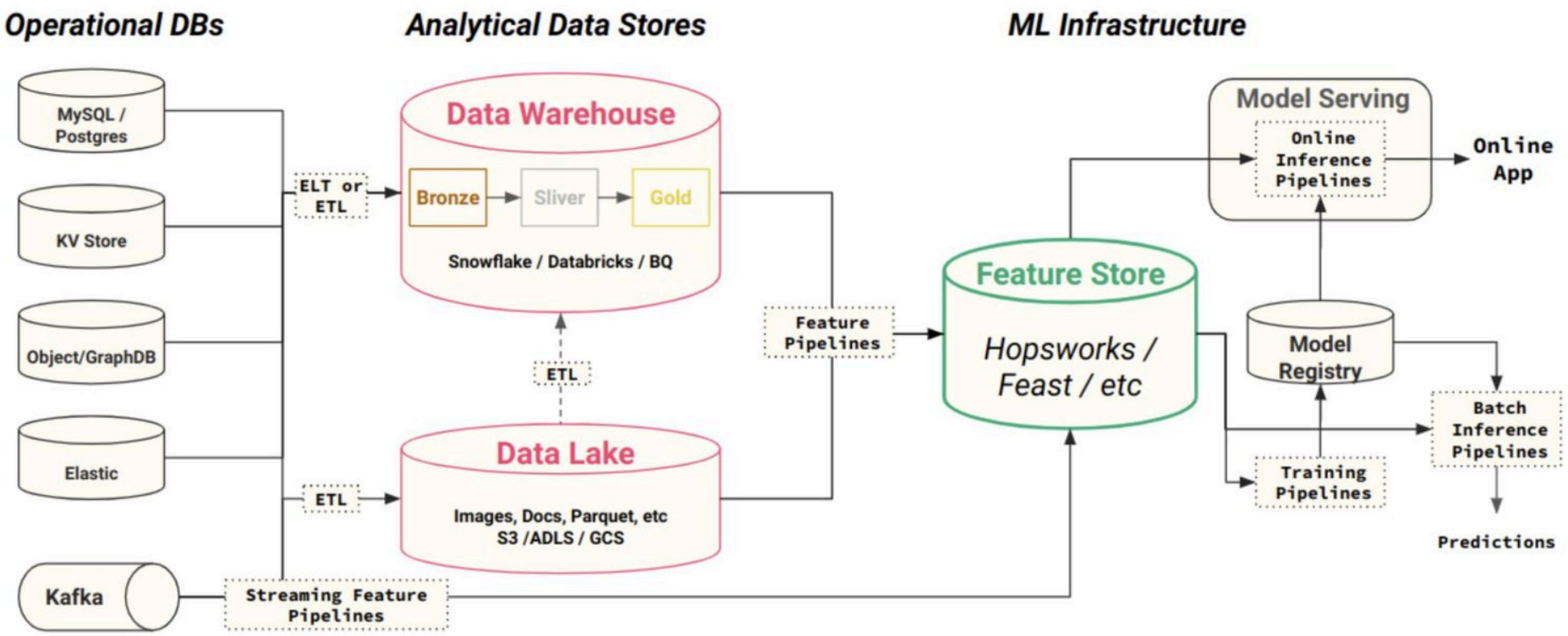
Feature, training, and inference pipelines can run independently




Shared stores connect pipelines with different schedules

# ML Infrastructure

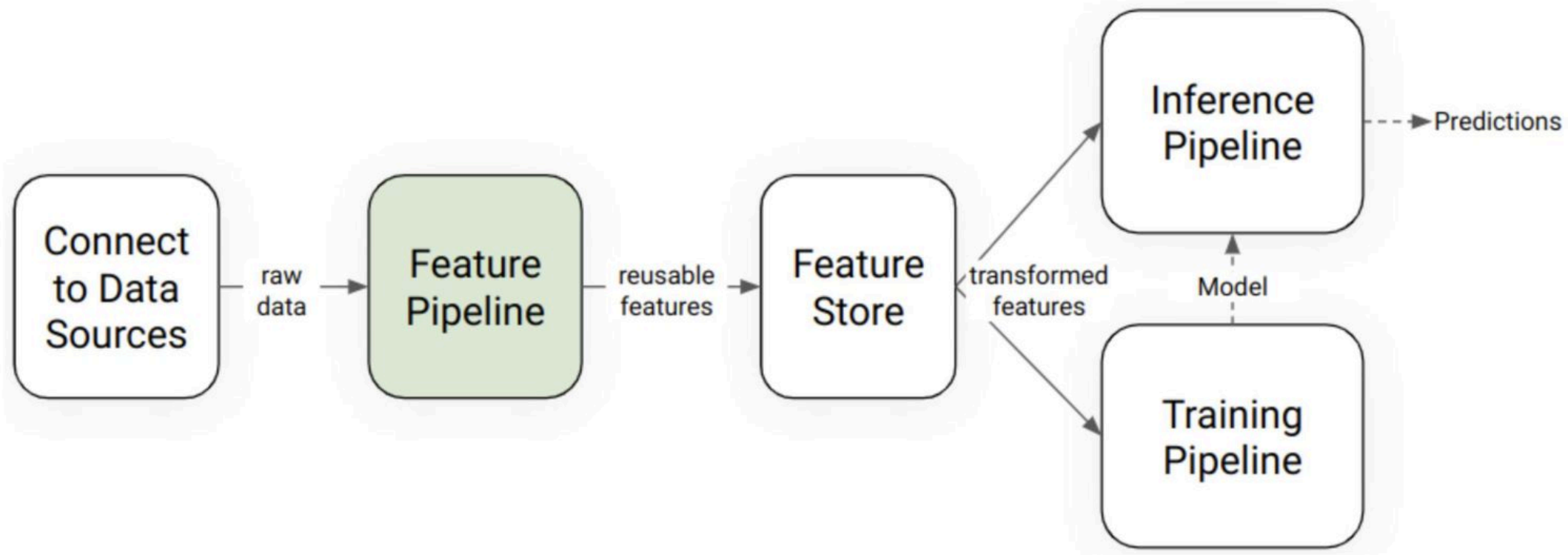
Operational data feeds analytical stores and ML pipelines




ML systems connect operational and analytical data

# Feature Pipeline

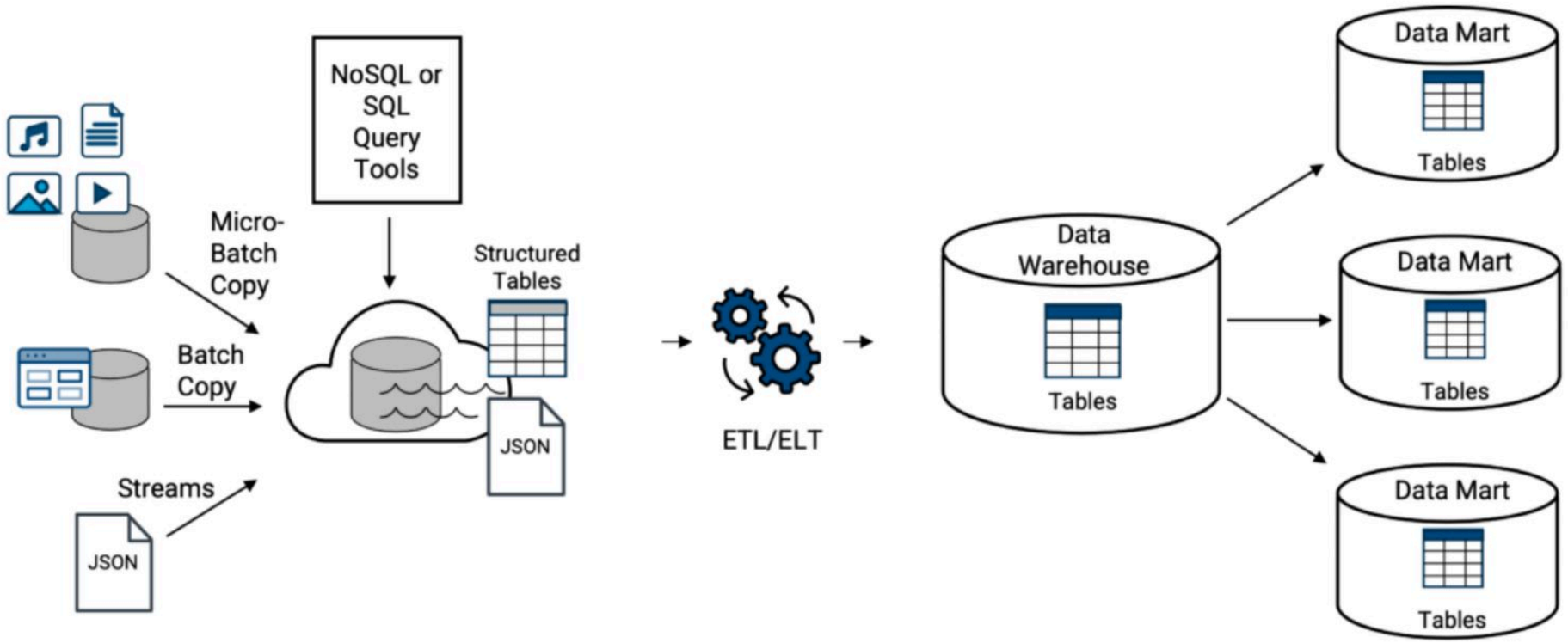
A feature pipeline creates model inputs from source records




Feature computation precedes both training and serving

# Recap: Data Preparation

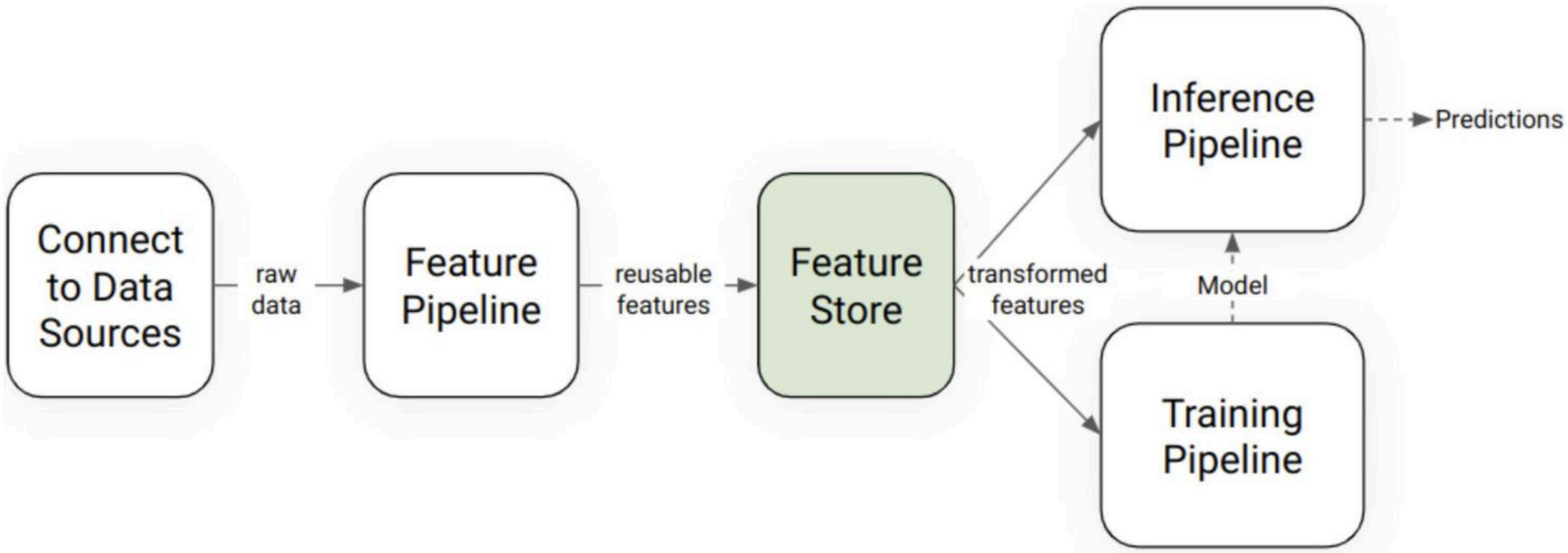
Model features depend on the same collection and preparation decisions




Raw sources become **structured tables** before features

# Feature Store

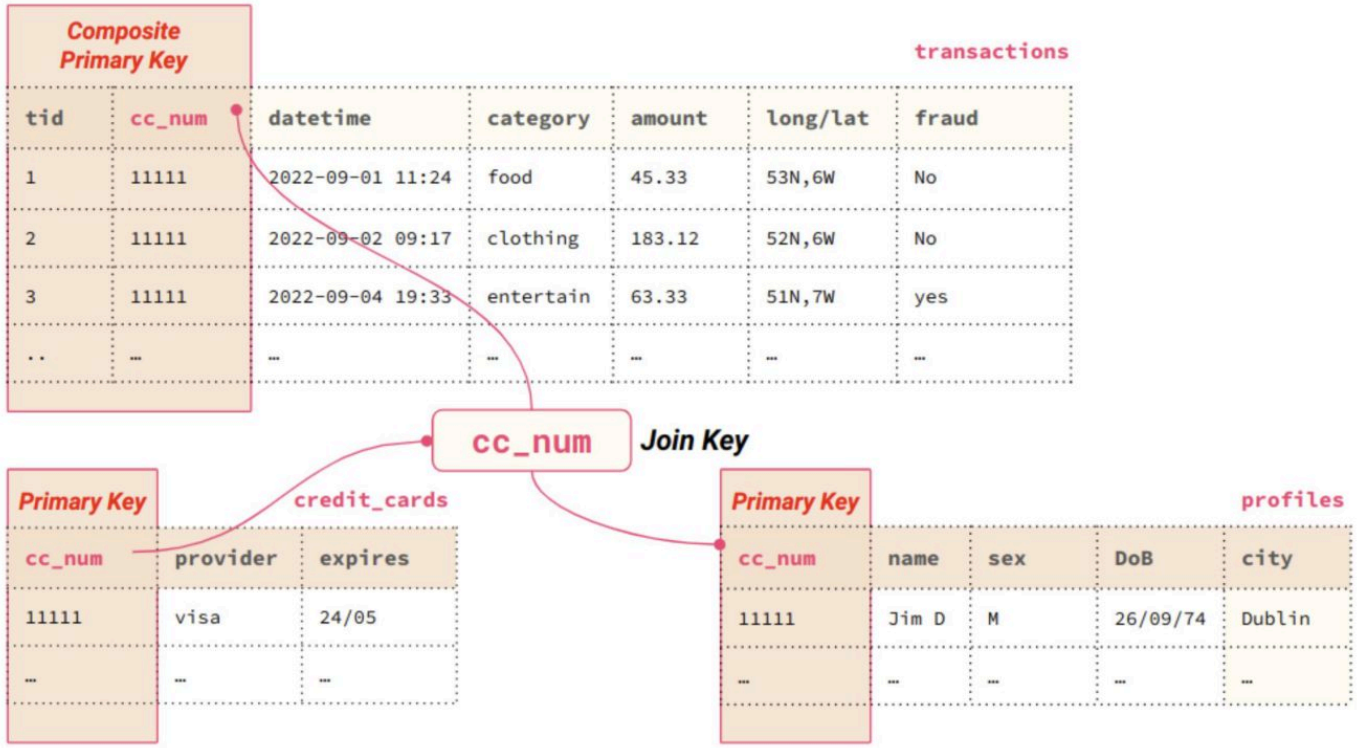
A feature store manages reusable feature values and definitions




A shared feature store feeds training and inference

# Feature Tables

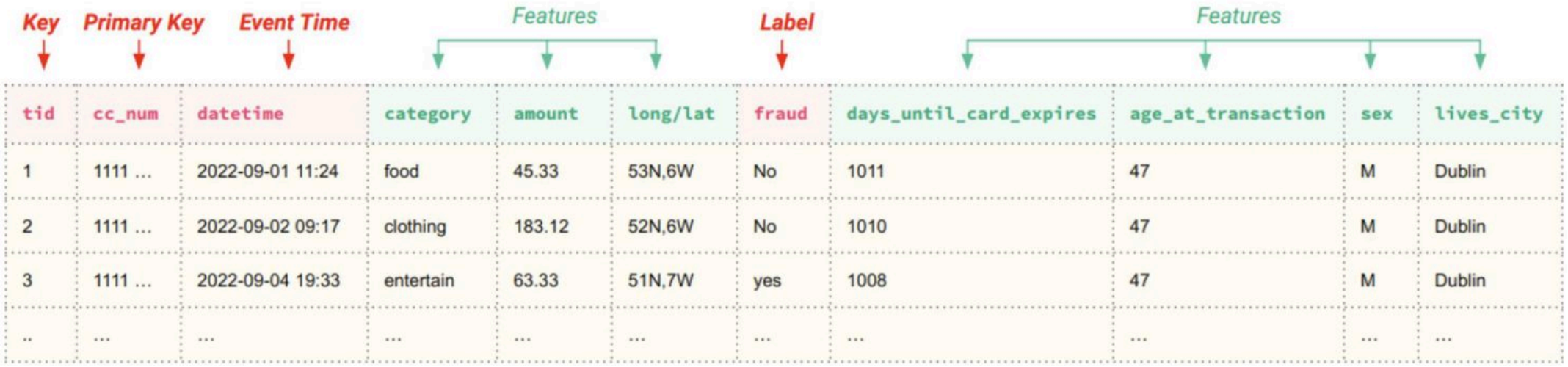
Entity keys connect features from different source tables




Joins align transaction, user, and credit features

# Training Table

A training example combines features with a target label




| tid | cc_num | datetime | category | amount | long/lat | fraud | days_until_card_expires | age_at_transaction | sex | lives_city |
|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 1111 ... | 2022-09-01 11:24 | food | 45.33 | 53N,6W | No | 1011 | 47 | M | Dublin |
| 2 | 1111 ... | 2022-09-02 09:17 | clothing | 183.12 | 52N,6W | No | 1010 | 47 | M | Dublin |
| 3 | 1111 ... | 2022-09-04 19:33 | entertain | 63.33 | 51N,7W | yes | 1008 | 47 | M | Dublin |
| .. | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... |

One row contains the features available for one prediction

# Feature Update Frequencies

Different feature groups can refresh at different rates

Our main Feature group is `transactions`. The columns for this one big table (OBT) have different data sources, that are updated at different cadences. It is inefficient to update all columns in every update, but less JOINs will be required for training data.

*Updated Frequently* | *Updated Less Frequently*

| Source: cc_transactions | | | | | | | Source: credit_card | Source: profile | | |
|---|---|---|---|---|---|---|---|---|---|---|
| tid | cc_num | datetime | category | amount | long/lat | fraud | days_until_card_expires | age_at_transaction | sex | lives_city |
| 1 | 1111 ... | 2022-09-01 11:24 | food | 45.33 | 53N,6W | No | 1011 | 47 | M | Dublin |
| 2 | 1111 ... | 2022-09-02 09:17 | clothing | 183.12 | 52N,6W | No | 1010 | 47 | M | Dublin |
| 3 | 1111 ... | 2022-09-04 19:33 | entertain | 63.33 | 51N,7W | yes | 1008 | 47 | M | Dublin |
| .. | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... |

A recent transaction and a user profile age differently

# Windowed Features

Aggregations summarize events over a defined time window

**`transactions_4h_aggs`** contains aggregated features computed over a 4h time window for each credit card

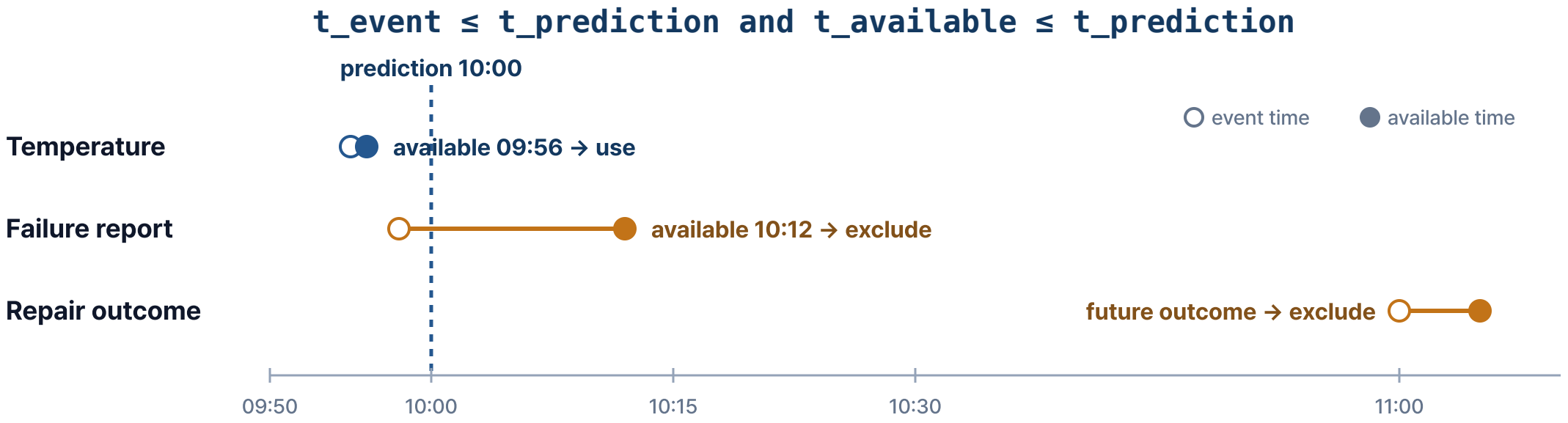



| cc_num | datetime | loc_delta_mavg | trans_volume_mstd | trans_volume_mavg | trans_freq |
|---|---|---|---|---|---|
| 1111 ... | 2022-09-01 11:24 | 53N,6W | 3.4 | 8 | 6 |
| 1111 ... | 2022-09-02 09:17 | 52N,6W | 3.6 | 3 | 5 |
| 1111 ... | 2022-09-04 19:33 | 51N,7W | 4.1 | 33 | 45 |
| ... | ... | ... | ... | ... | ... |

The aggregation window is part of the feature definition

# Point-in-Time Correctness

Reconstruct inputs using what the system knew at prediction time




Use a record only if it was **available** at 10:00

# Recap: Feature Availability

A late report existed in event time but was unavailable at prediction time

| Feature | Event | Available | Value |
|---|---|---|---|
| temperature | 09:55 | 09:56 | 31 |
| alarm_count | 09:58 | 10:12 | 3 |
| vibration | 09:50 | 09:51 | 0.8 |

A late report existed in event time but was unavailable at prediction time

# Demo: Point-in-Time Features

Compare event-time filtering with availability-time filtering

1. Join Predictions and Features using machine_id; filter event_at ≤ predicted_at.
2. Also filter available_at ≤ predicted_at. Which feature disappears?

```
-- write your query
here, then Run
```

Run ▶ Reset Ctrl/⌘ + Enter

# Answer: Point-in-Time Features

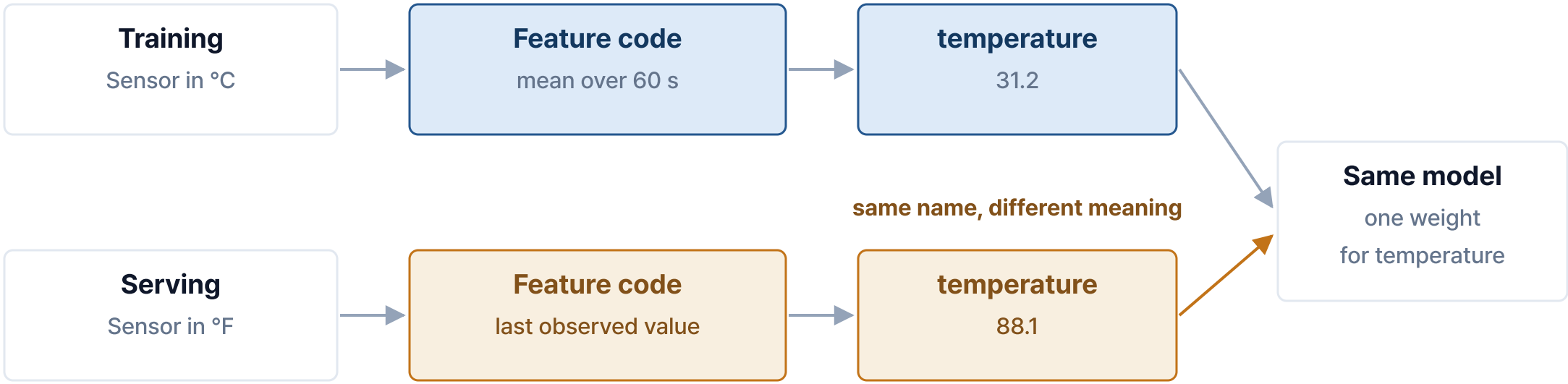
Use only features that were available when the prediction was made

```
SELECT p.id, f.feature, f.value
FROM Predictions p JOIN Features f ON
f.machine_id=p.machine_id
WHERE f.event_at <= p.predicted_at AND
f.available_at <= p.predicted_at
ORDER BY p.id, f.feature;
```

The late alarm report is excluded

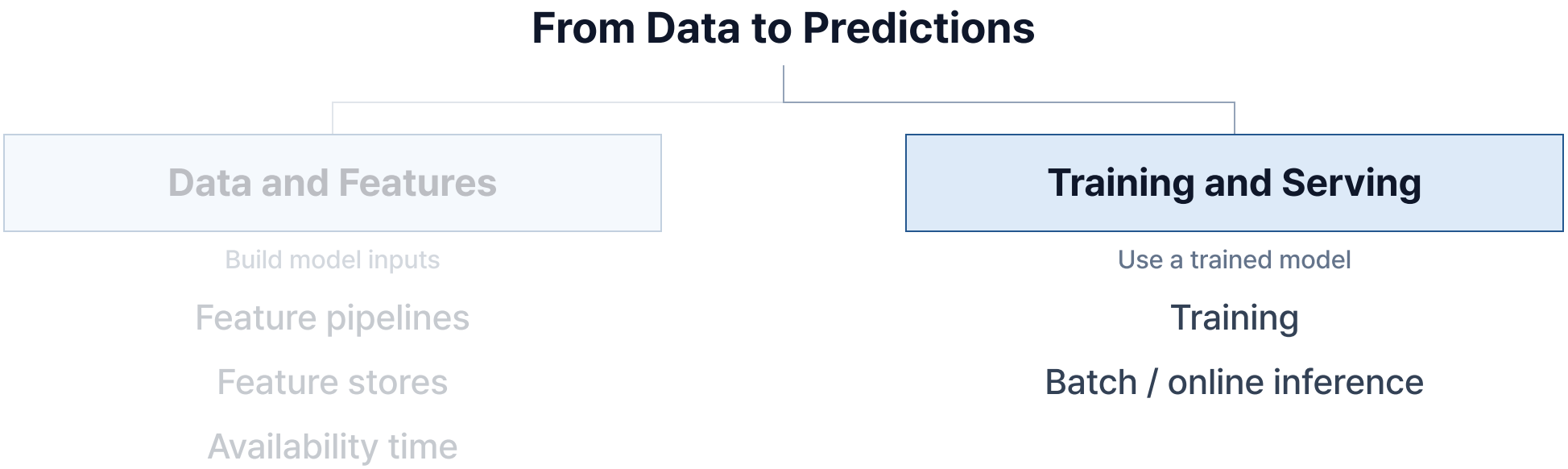
# Caveat: Training–Serving Skew

Different feature code can change the meaning of an input



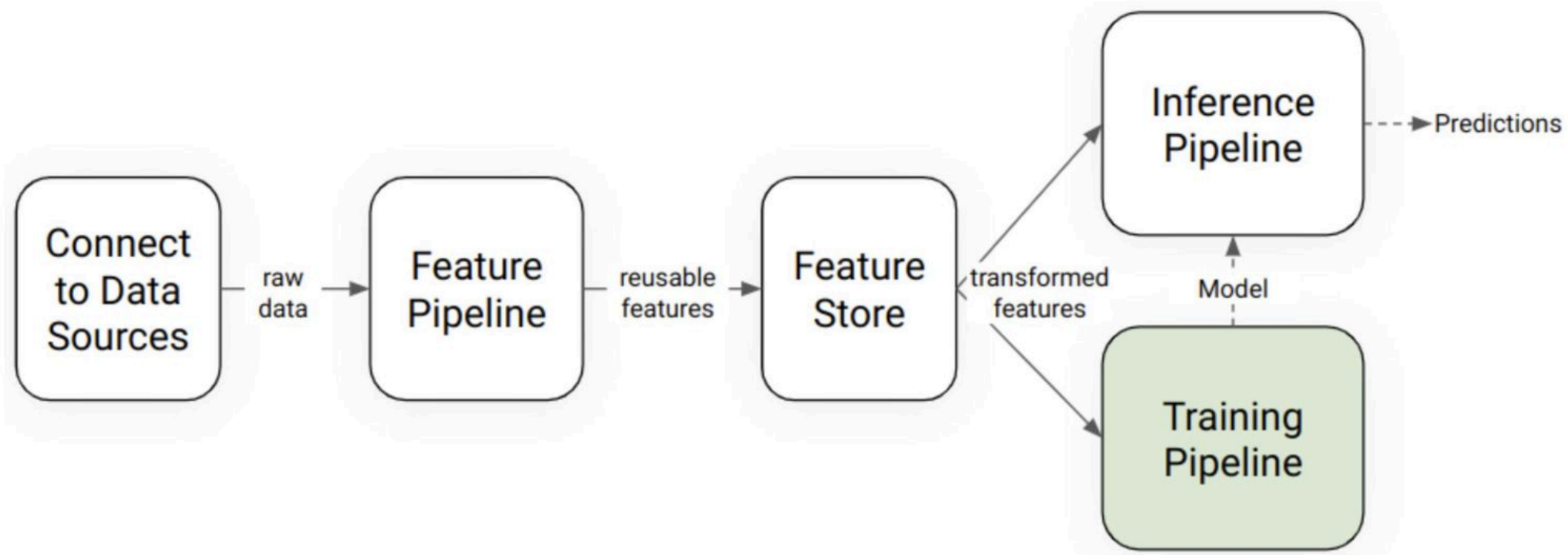

Same feature name, **different computation**

# § Training and Serving



# Training Pipeline

A training pipeline reads versioned examples and produces an artifact




The model artifact leaves the training pipeline

# Train, Validation, and Test

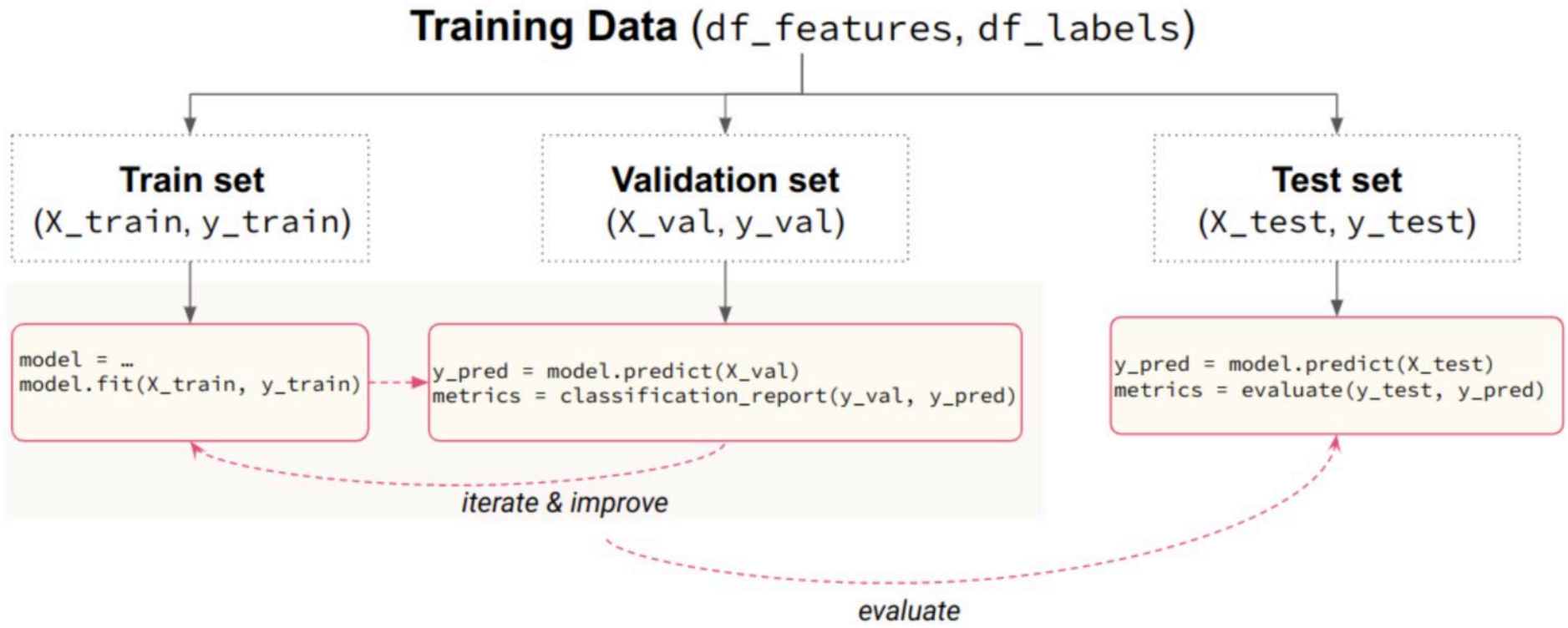
Use validation for choices and reserve test data for final evaluation




The original 70/15/15 split is an example, not a rule

# Training Iteration

Repeated experiments consume the same controlled evaluation protocol




Training and validation guide the iteration

# Experiment Record

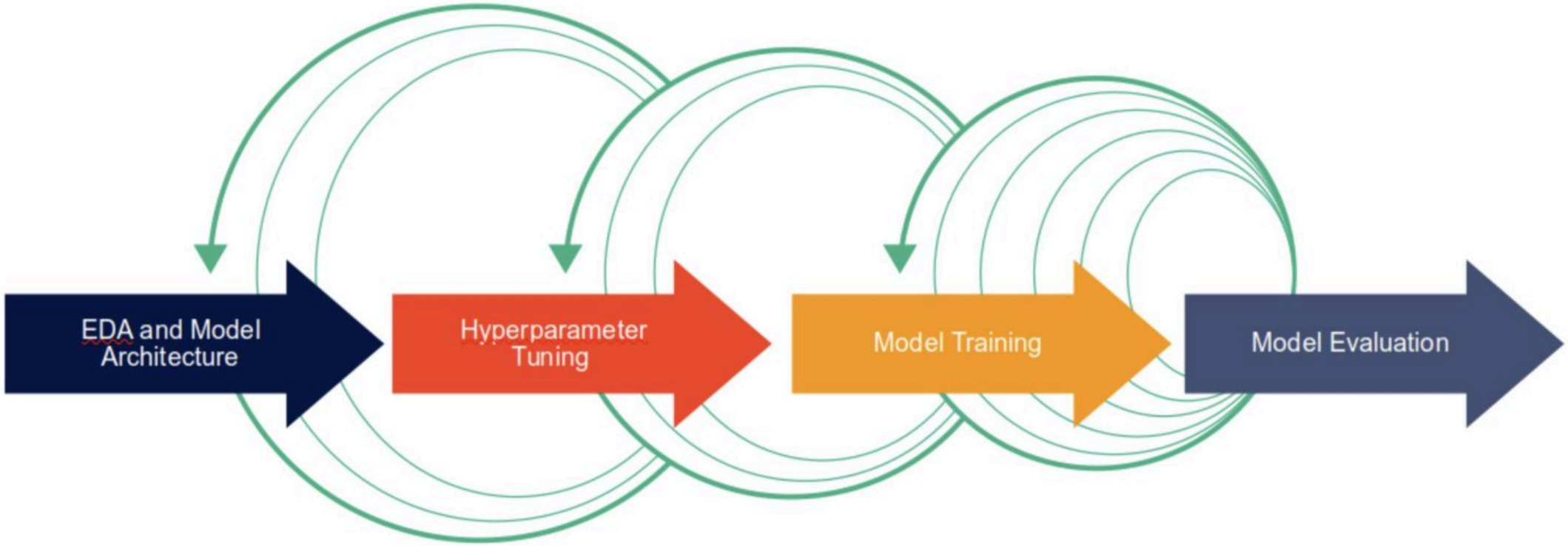
Record the inputs that produced each candidate model




The record names every input that **built** the model

# Experiment Cycle

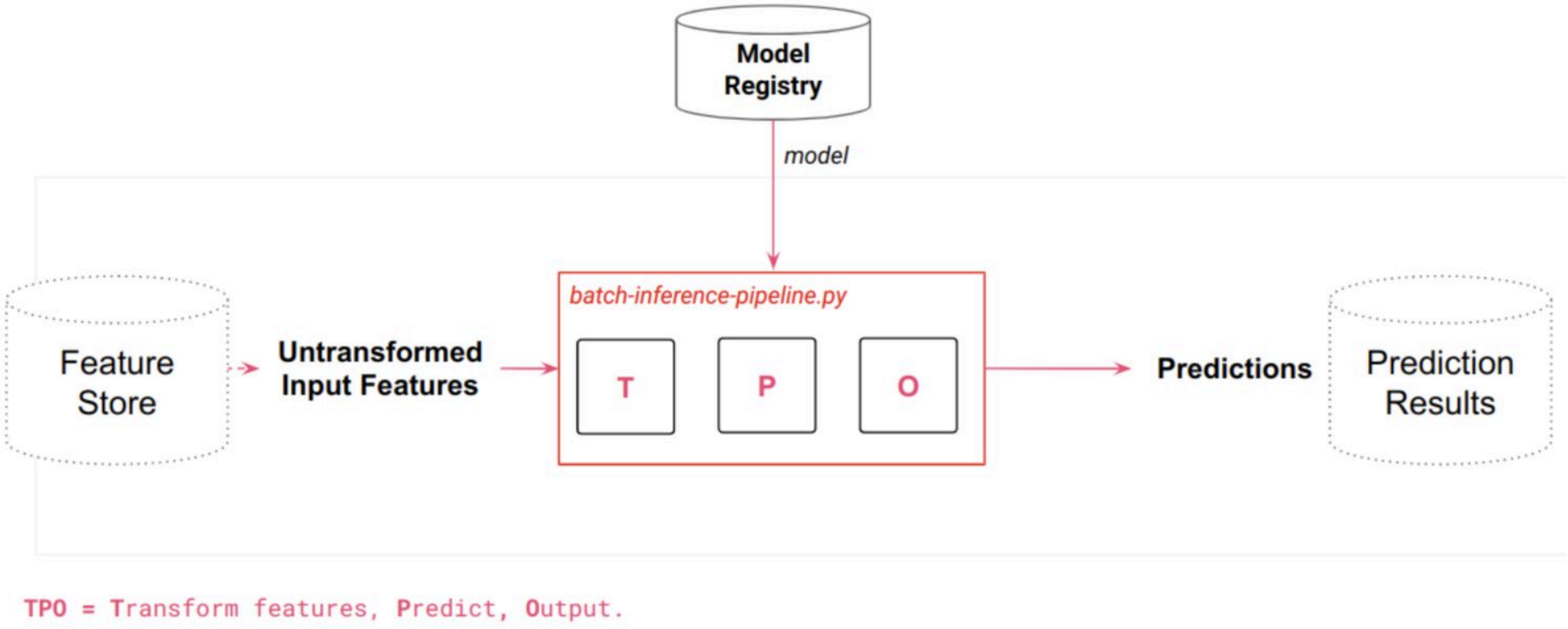
Try, evaluate, and revise before promoting a candidate




Iteration changes the candidate, not the held-out test

# Batch Inference

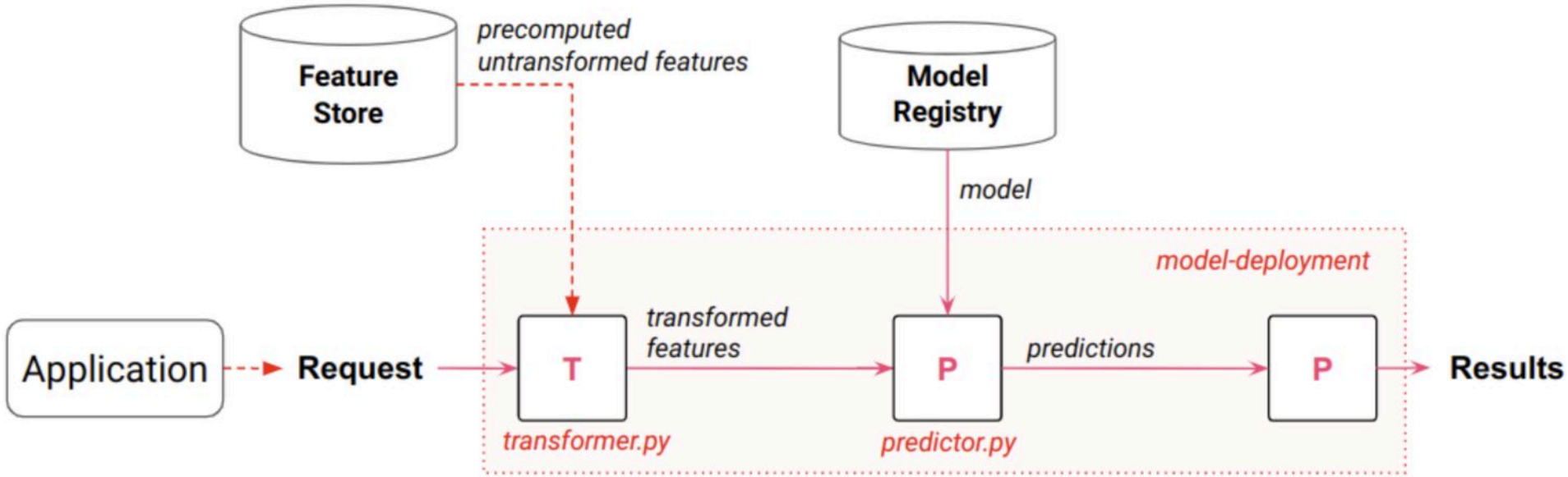
Batch inference scores a collection of records on a schedule




Features enter a model and predictions return to storage

# Online Inference

Online inference answers an individual request within a latency budget

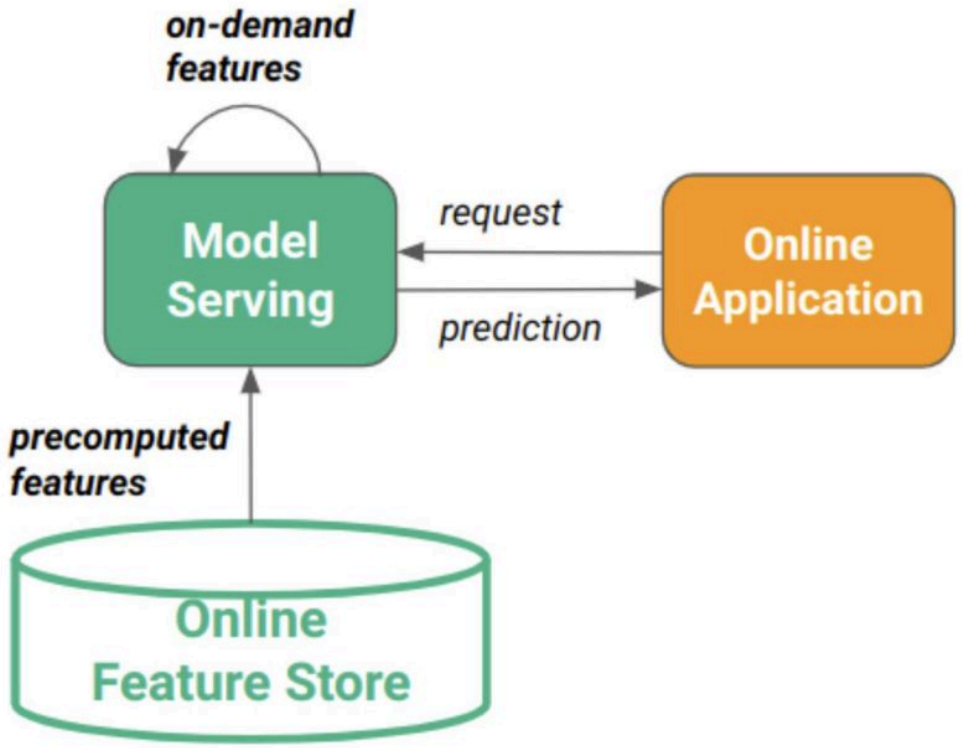



TPP = Transform the input request into features, Predict using input features and the model, Post-process predictions, before output results.

A request joins current features with the deployed model

# Real-Time ML System

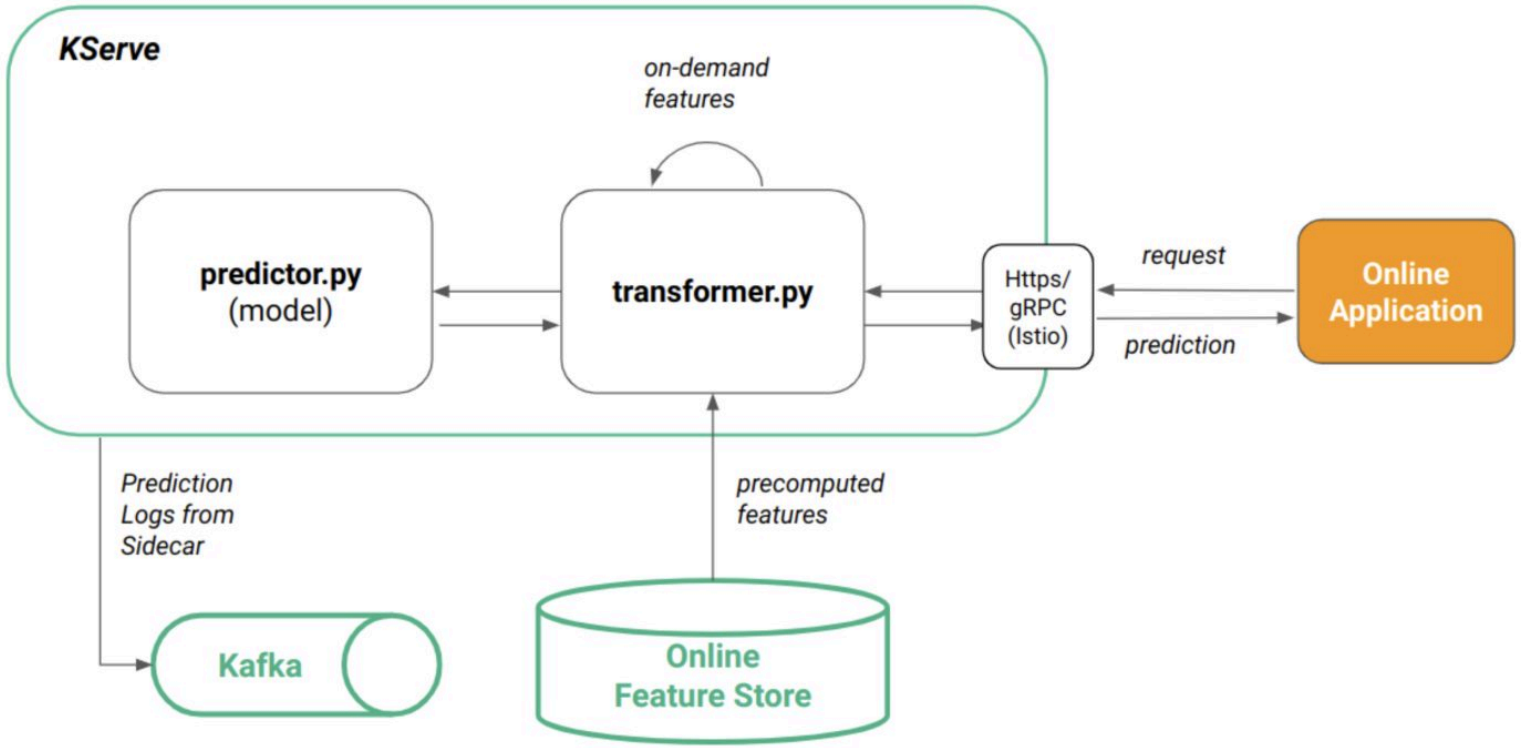
Low-latency features and model serving are separate components




Feature freshness and response latency are different requirements

# Model Serving Components

A transformer and predictor can share a serving endpoint




Transformation and prediction remain observable stages

# 02 · Building and Releasing Models

01

## From Data to Predictions

prepare inputs and use a trained model

02

## Building and Releasing Models

check models before putting them into use

03

## Monitoring Models in Use

notice changes and update the model

# § Development

## Building and Releasing Models

**Development**

Create versioned artifacts

DevOps / MLOps

Reproducibility

**Validation**

Check release readiness

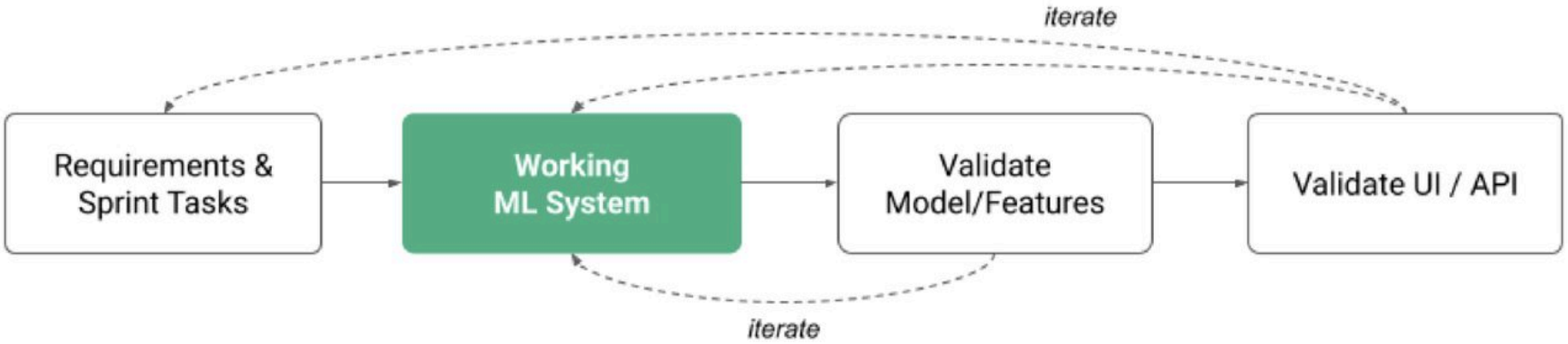
Data / model tests

Release gates

# MLOps

MLOps coordinates development, deployment, and monitoring of ML systems

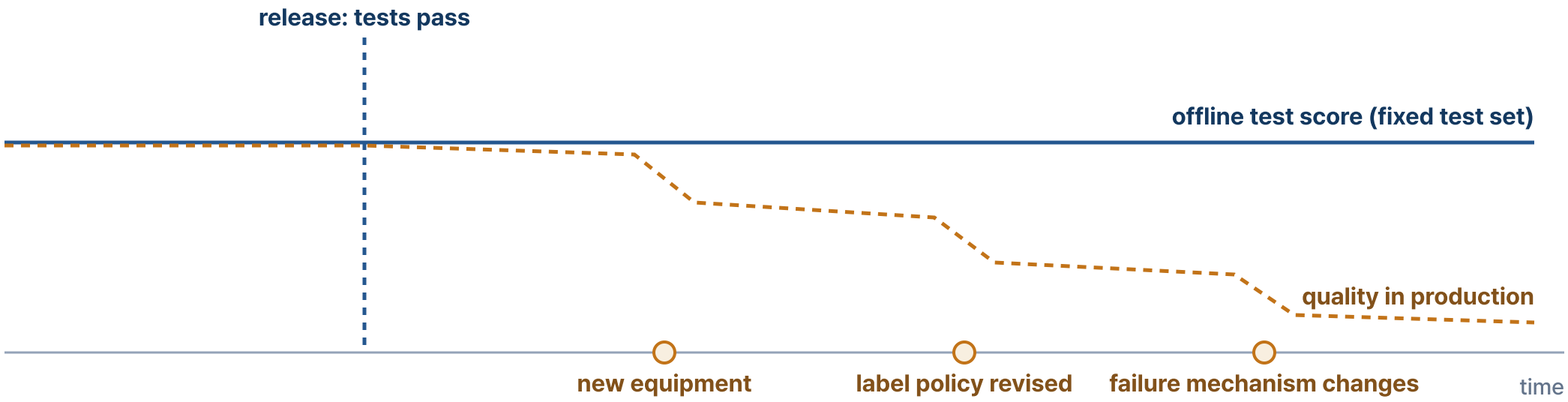
- A goal of MLOps is to improve both iteration speed and quality when developing ML




The deployed system returns observations to development

# Changing ML Products

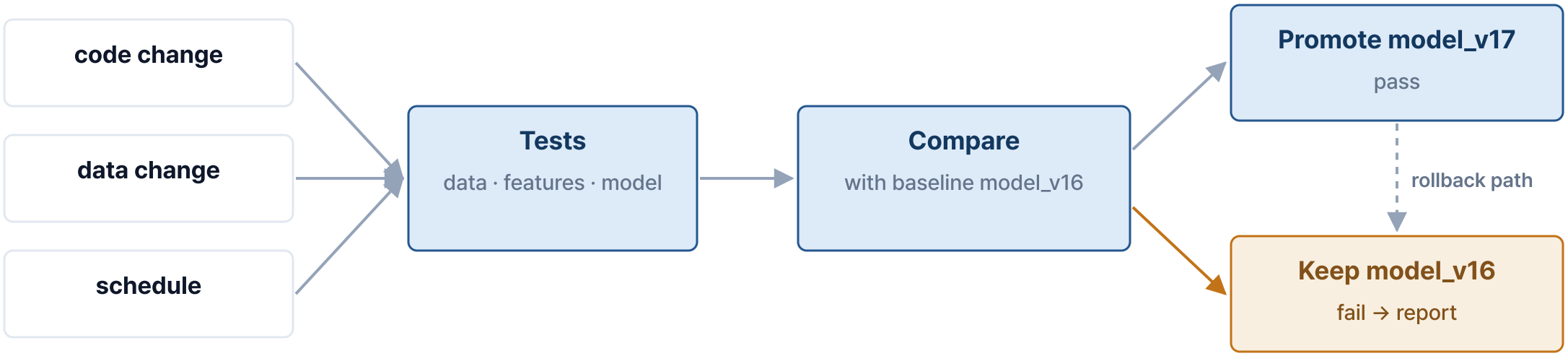
Data and targets can change after deployment




The world keeps changing **after** the release test

# Release Automation

Automation connects repeatable tests to controlled releases

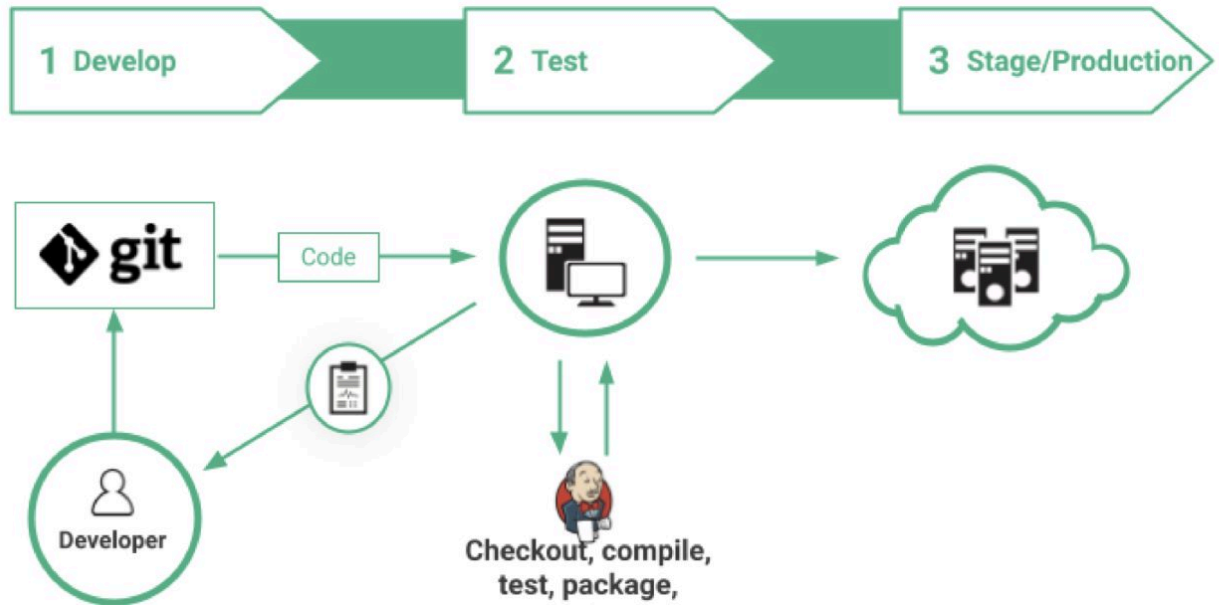



Every change passes the same tests and **baseline** comparison

# DevOps

DevOps automates the software build, test, and release lifecycle

- DevOps is a set of practices, tools, and a cultural philosophy that automate and integrate the processes between software development and IT teams. Key technologies are **version control, automated testing, versioning** of production deployments.

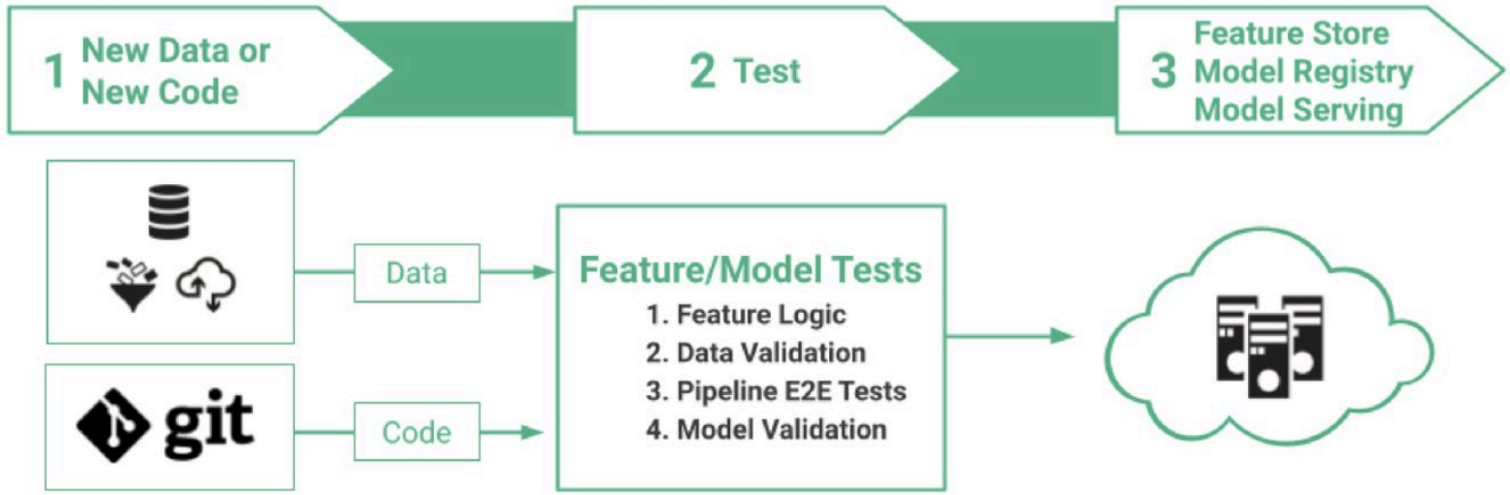



Versioned code enters a tested release pipeline

# DevOps and MLOps

Data and model changes can trigger ML validation and retraining

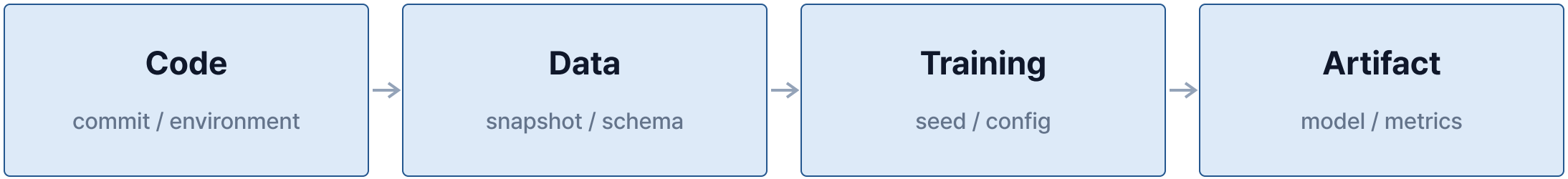
- In DevOps, changes in source code trigger automated testing and deployment.
- In MLOps, changes in either source code or incoming data trigger automated testing and deployment.




ML pipelines respond to more than code commits

# Reproducible ML

Reproduction needs more than a saved model

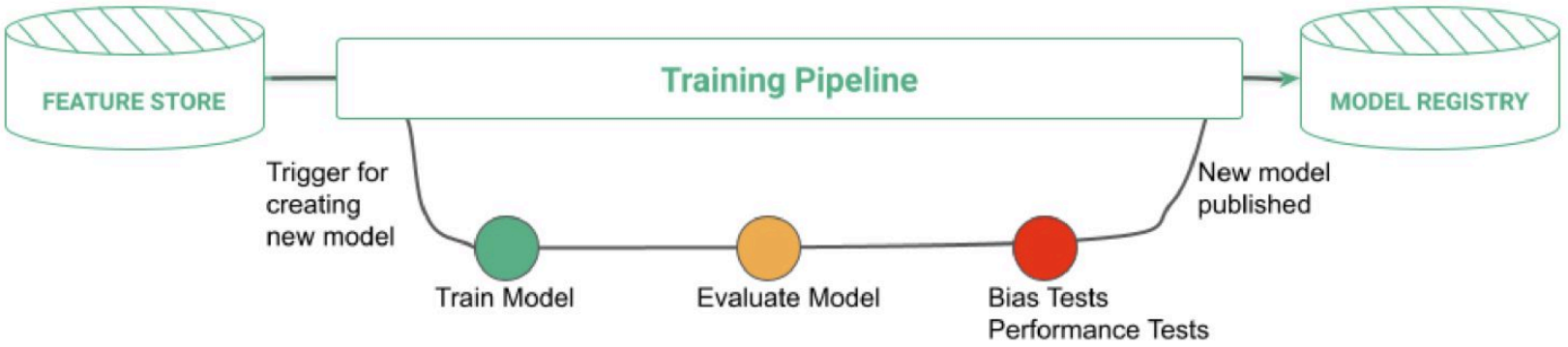



Code → Data → Training → Artifact

# Model Development

Train and test on separate data before deployment

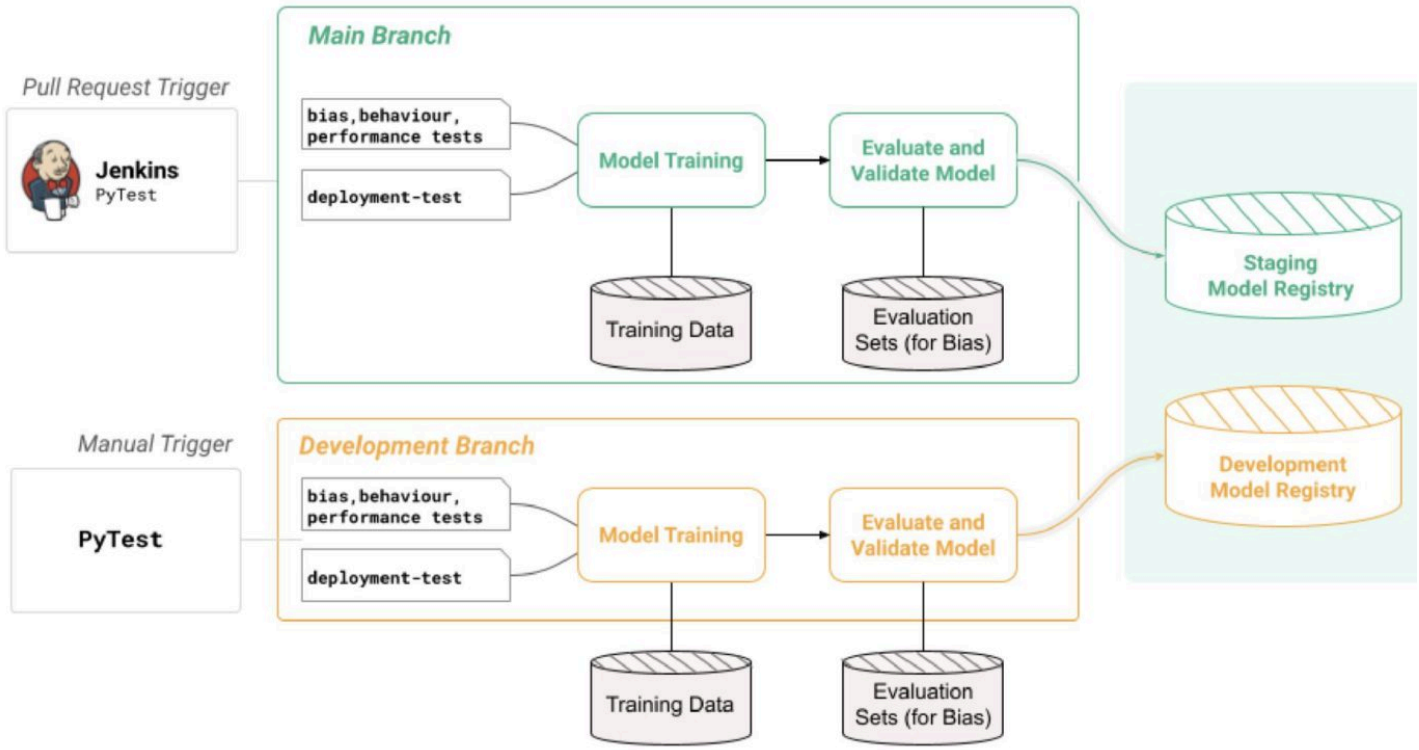
- Save the hyperparams and training data used to train the model,
- Only deploy a model that outperforms an existing model deployment (or baseline), assuming both trained on same dataset.
- Only deploy a model if it is free from bias and trained on ethical data.




A candidate passes a test before the serving pipeline

# Development Environments

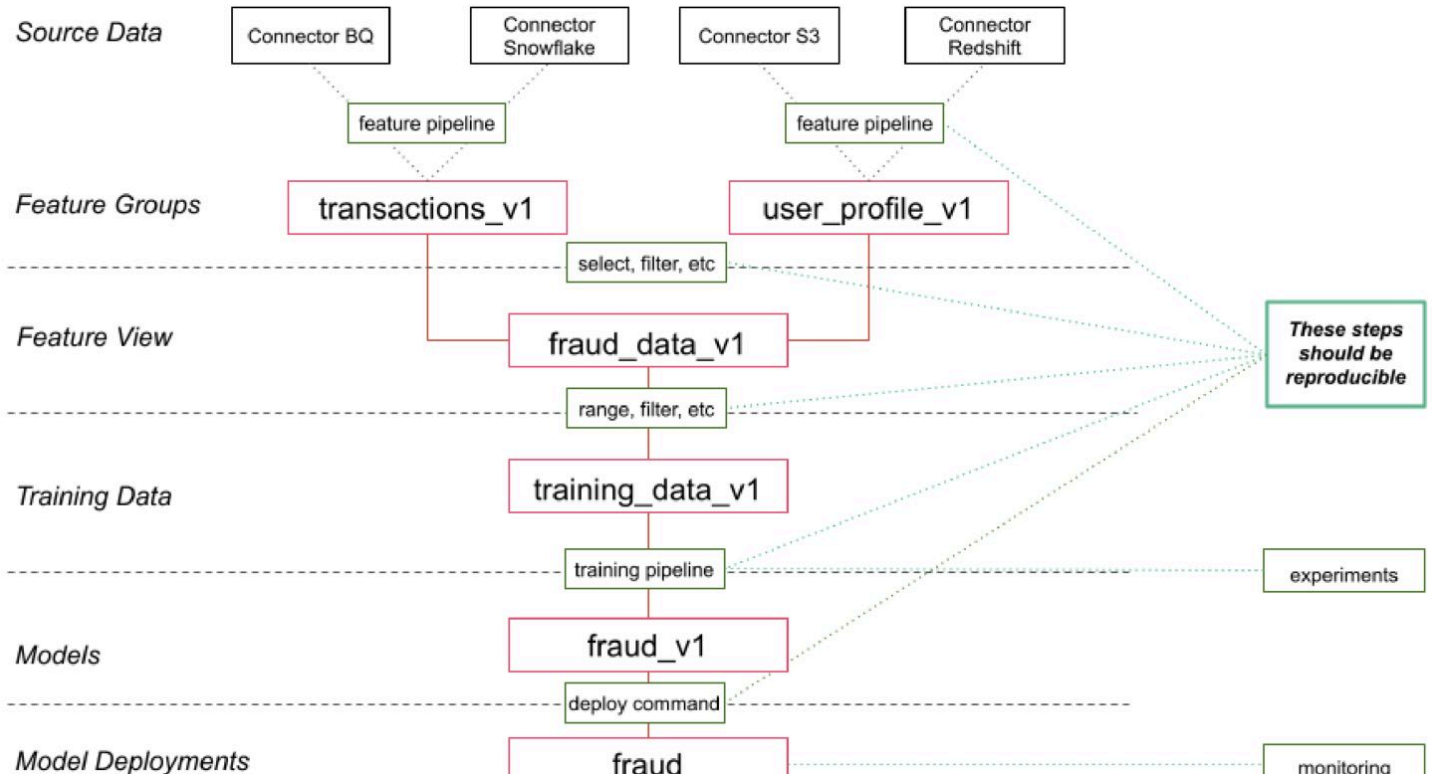
Development and production require explicit artifact promotion




Move versioned artifacts across environment boundaries

# Reproducibility Dependencies

Code, feature groups, and training data form a dependency graph




A model version points back to its inputs

# Versioned ML Assets

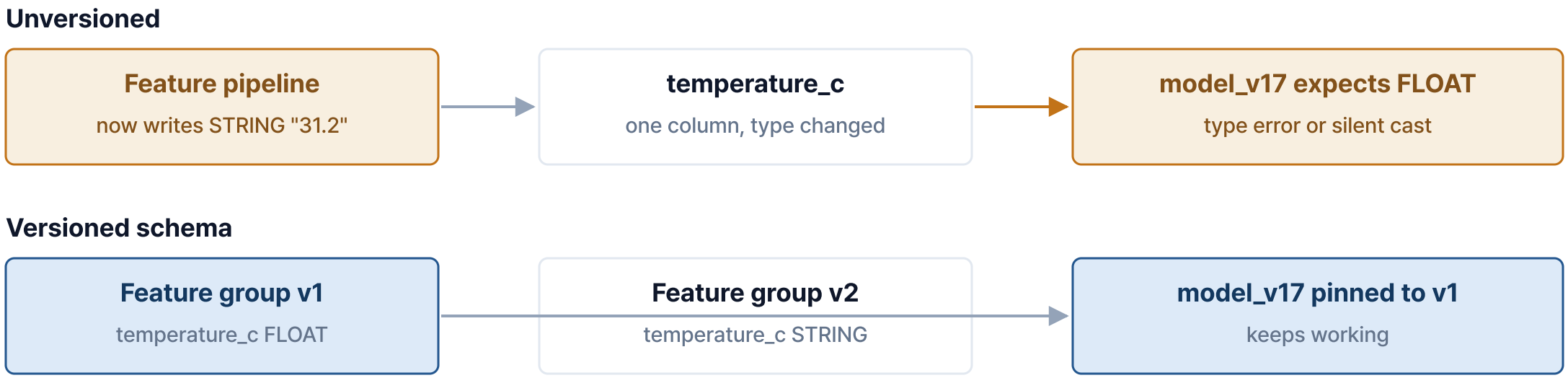
Choose a versioning mechanism for each asset

| Asset | Version key | Reproduction input |
| --- | --- | --- |
| Code | Git commit | Code + environment |
| Features | Definition + schema | Transformation + snapshot |
| Training data | Snapshot ID | Rows + split |
| Model | Artifact ID | Parameters + dependencies |

Choose a versioning mechanism for each asset

# Schema Versioning

A renamed or retyped feature can break a deployed model




A pinned schema version keeps the **deployed model** working

# Data Versioning

Large datasets need versioned references and reproducible reads

| Mechanism | Stores | Use |
| --- | --- | --- |
| Git | Code / small metadata | Track definitions |
| DVC-style tracking | Pointers + object storage | Track external files |
| Table snapshots | Versioned table state | Reproduce a query |

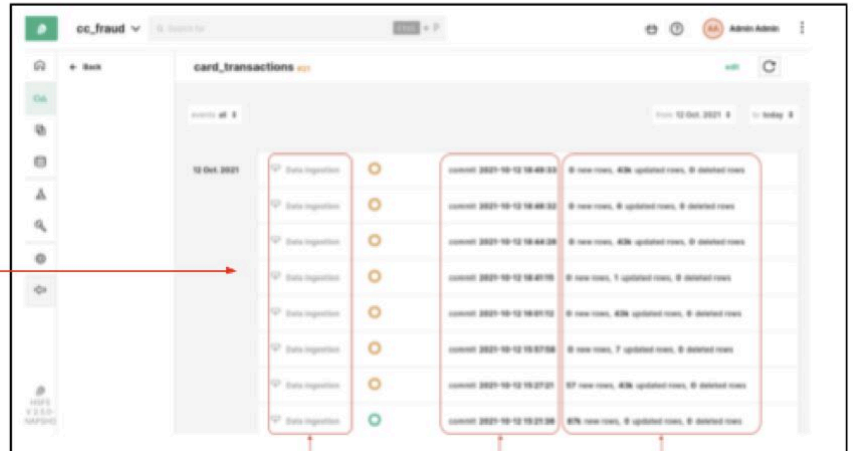
Large datasets need versioned references and reproducible reads

# Versioned Tables

A table snapshot can reproduce historical training inputs




View Commits in the Hopsworks UI in the “Activity” tab of a Feature Group



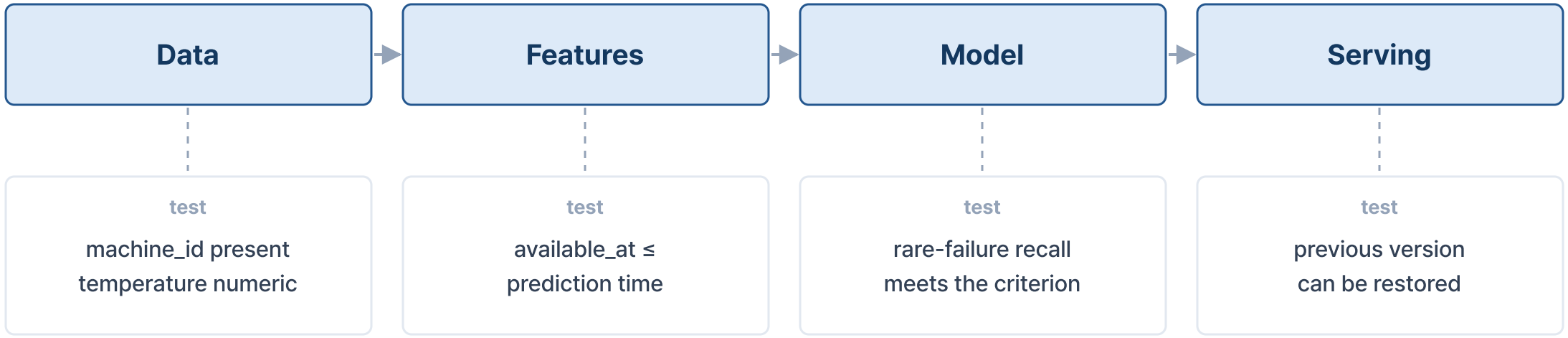

Versioned reads connect a training run to a data snapshot

# § Validation


Building and Releasing Models
Development
Create versioned artifacts
DevOps / MLOps
Reproducibility
Validation
Check release readiness
Data / model tests
Release gates

# ML Test Layers

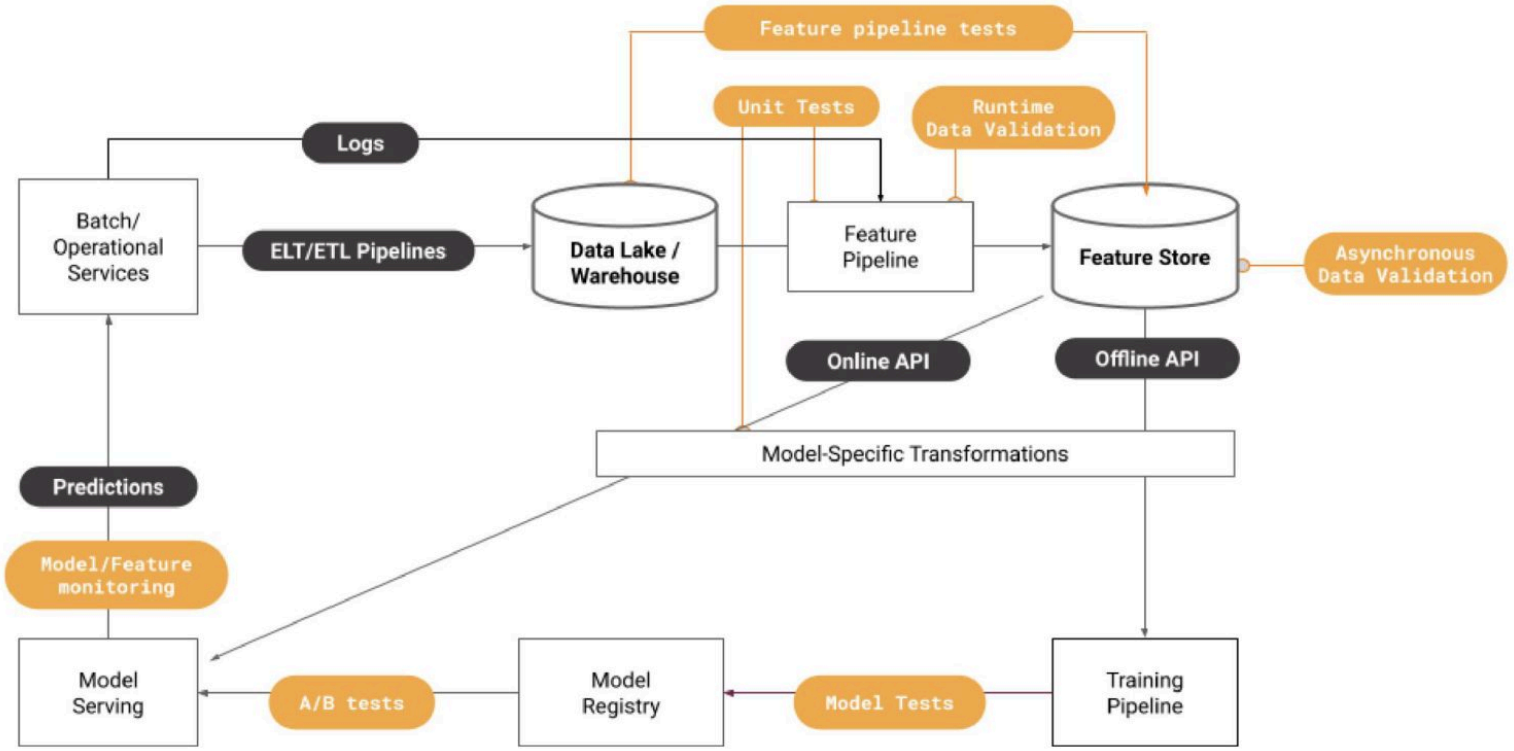
Each stage needs a test for its own failure modes




Each stage gets a test for **its own** failure

# Tests in ML Systems

Validation follows the dependencies from data to serving




Tests attach to the stage that owns the failure

# Data and Model Tests

Test contracts and quality with different assertions

| Test | Example assertion |
| --- | --- |
| Schema | machine_id exists; temperature is numeric |
| Data quality | Missing temperatures below a chosen threshold |
| Feature correctness | Feature available_at ≤ prediction time |
| Model quality | Recall on rare failures meets the release criterion |

Test contracts and quality with different assertions

# Infrastructure and Monitoring Tests

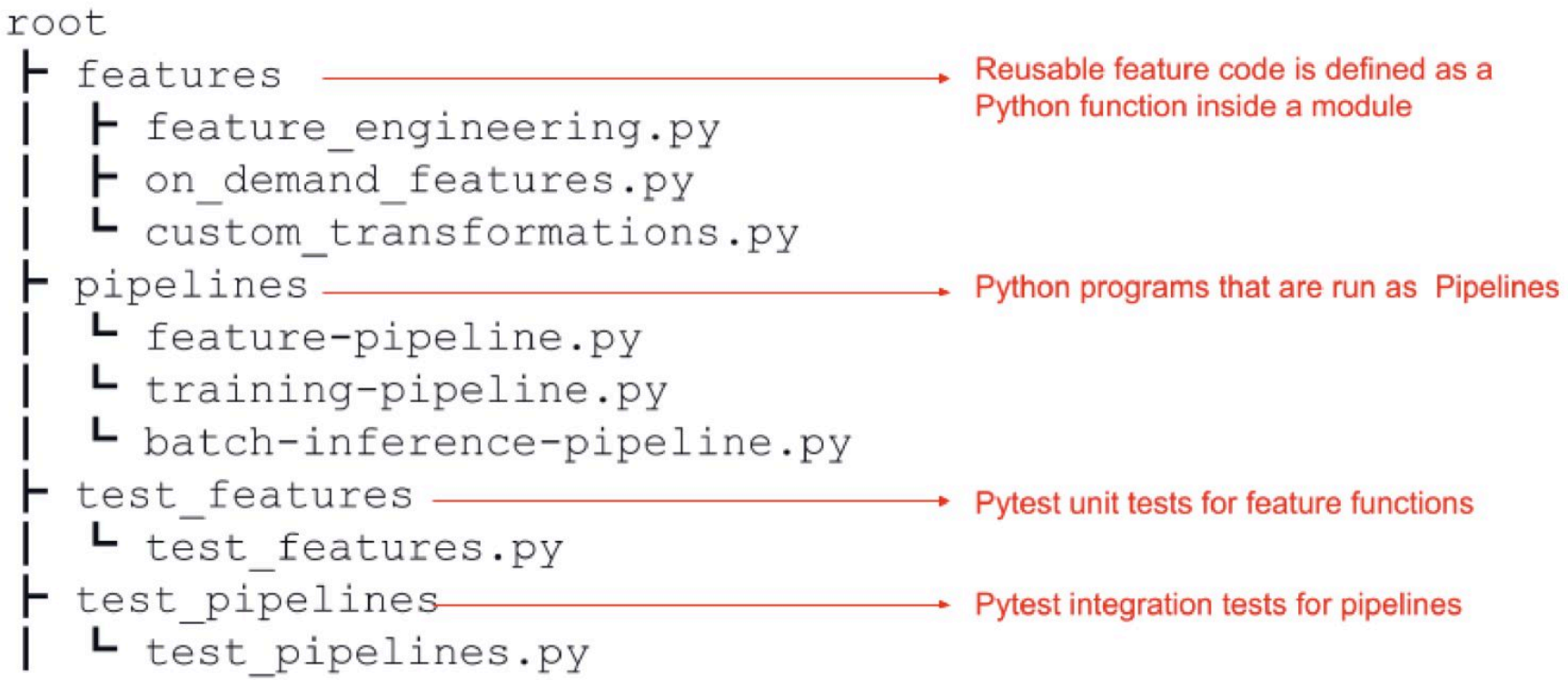
Test the pipeline as well as its output

| Layer | Example |
| --- | --- |
| Integration | A source batch reaches the scored table |
| Reproducibility | The recorded snapshot reproduces the feature rows |
| Rollback | The previous model version can be restored |
| Monitoring | A missing-label spike raises an alert |

Test the pipeline as well as its output

# Testable Feature Functions

Separate transformation logic from storage and orchestration




Feature logic lives in modules that **tests** import

# 03 · Monitoring Models in Use

01

## From Data to Predictions

prepare inputs and use a trained model

02

## Building and Releasing Models

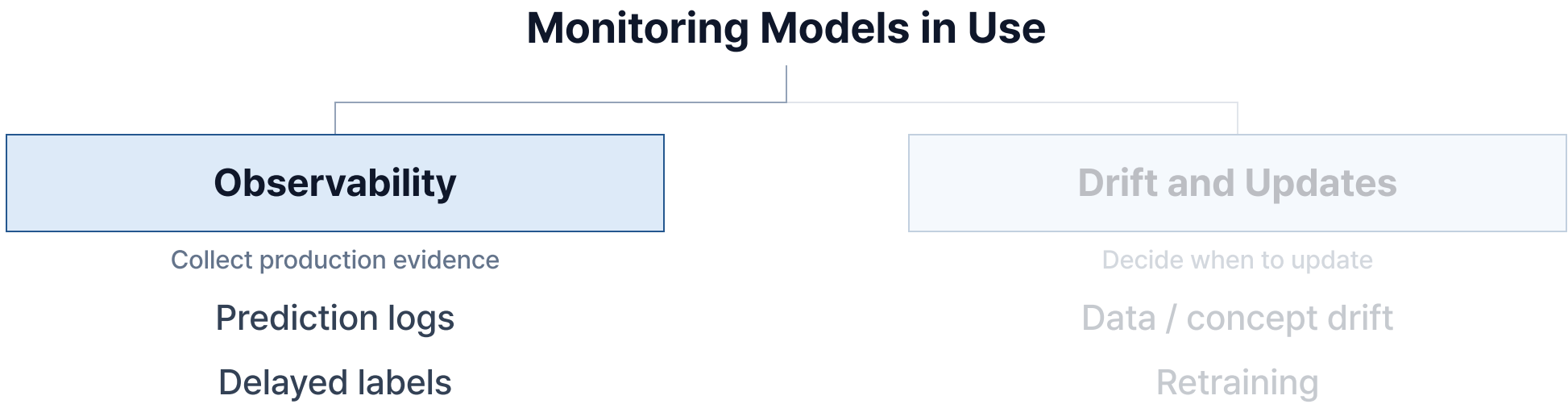
check models before putting them into use

03

## Monitoring Models in Use

notice changes and update the model

# § Observability



# Prediction Logs

Link each prediction to the model and feature versions

| request_id | machine | features | model | prediction | outcome |
|---|---|---|---|---|---|
| req-881 | P-17 | temp_v2 = 31.2 | model_v17 | fail · 0.82 | arrives day 3 |

identify the request

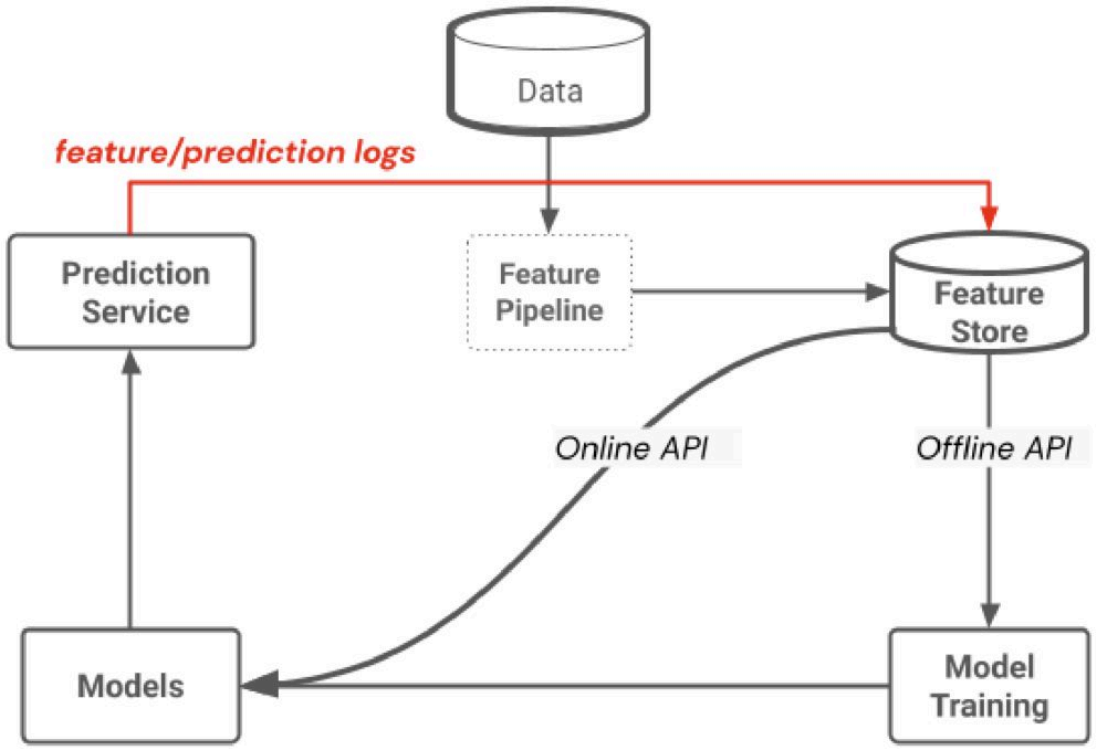
reproduce the prediction

evaluate later

One log row links request, inputs, model and **outcome**

# Feature and Prediction Logs

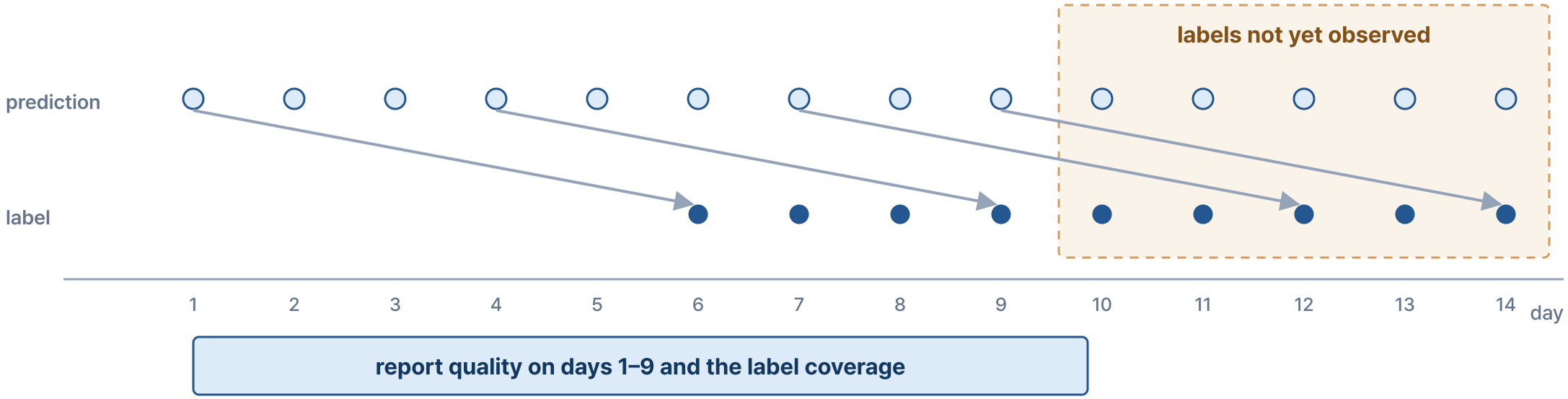
Logged inputs support debugging and later evaluation




Logs link input features, model versions, and outcomes

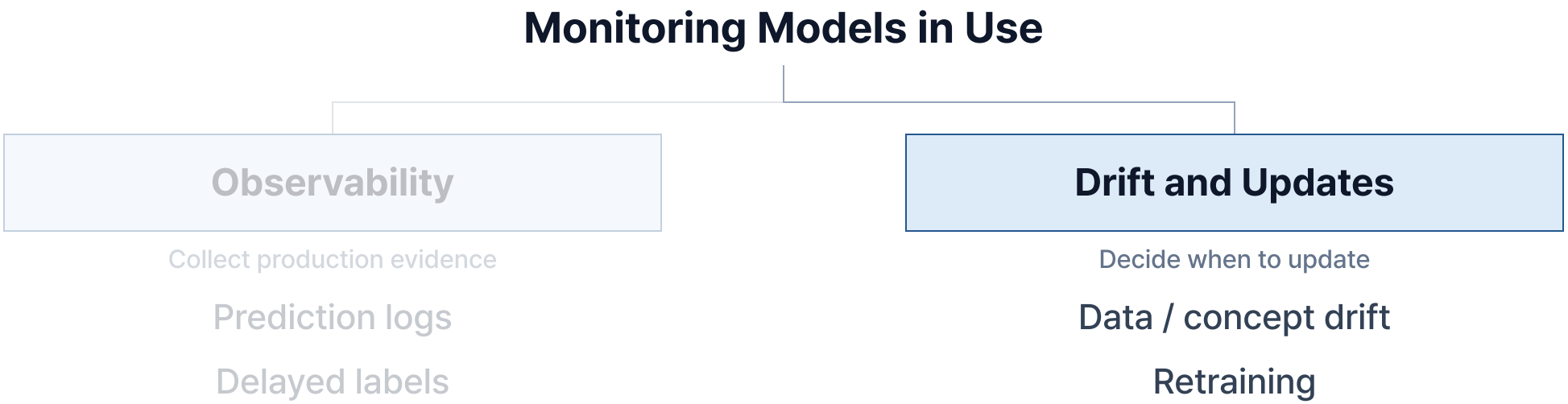
# Delayed Labels

Prediction quality becomes visible only when outcomes arrive



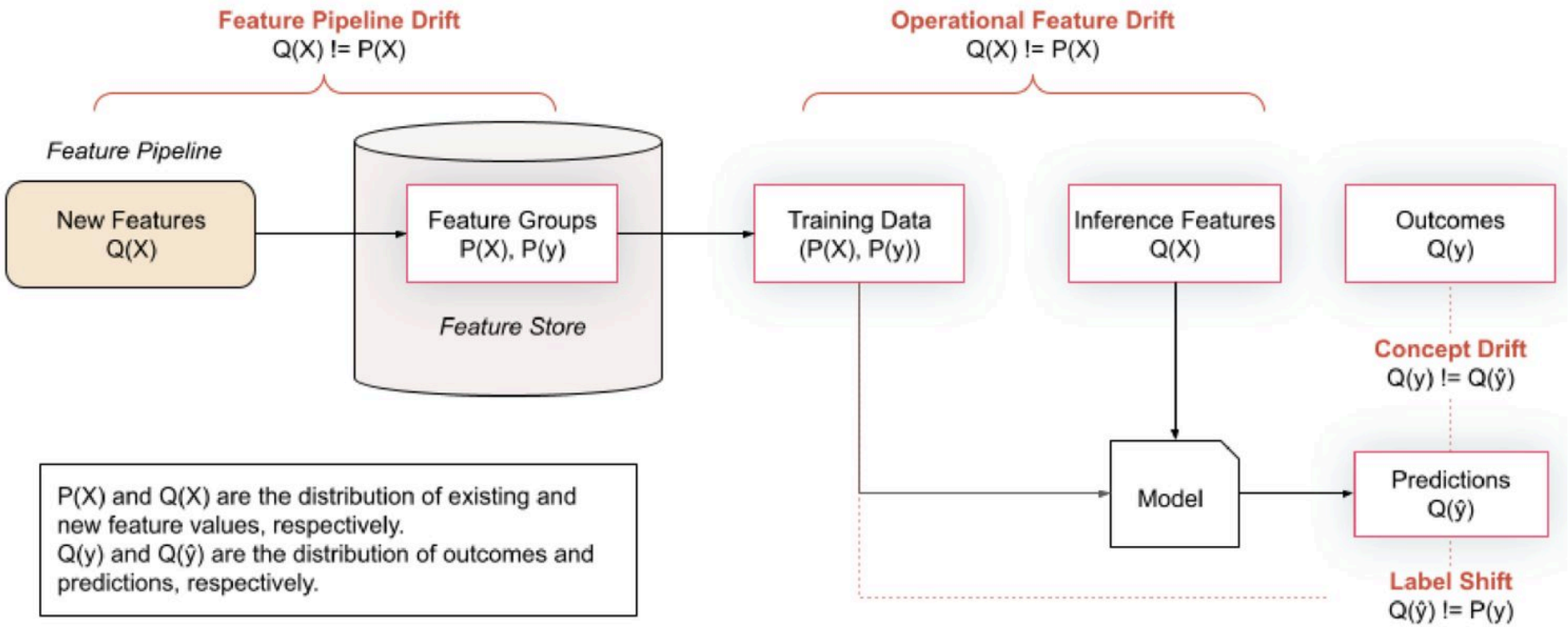

Recent predictions have **no outcome** yet

# § Drift and Updates



# Case study: Data Drift

Different drift locations call for different investigations



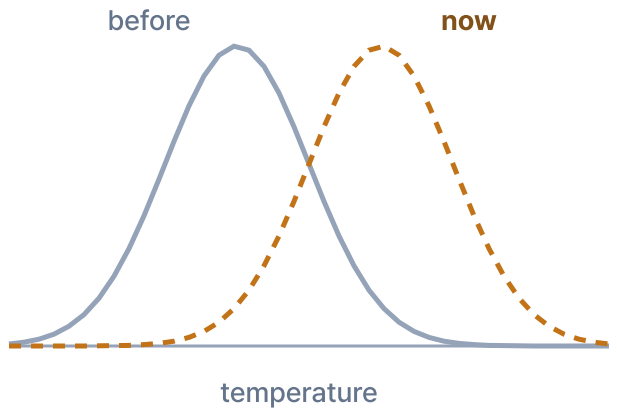

Feature-pipeline, serving, label, and concept drift differ

# Drift Types

Distribution change and predictive relationship change are distinct

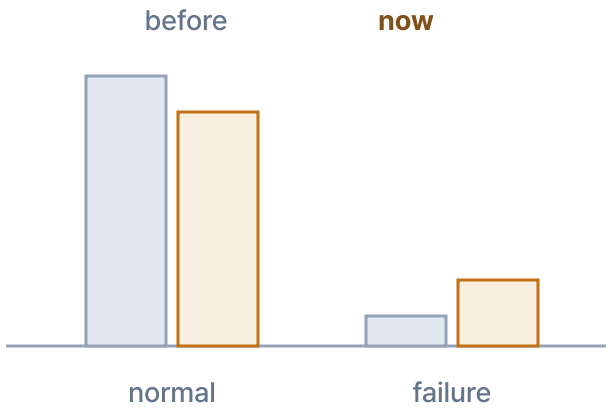
### Covariate shift

$P(x)$



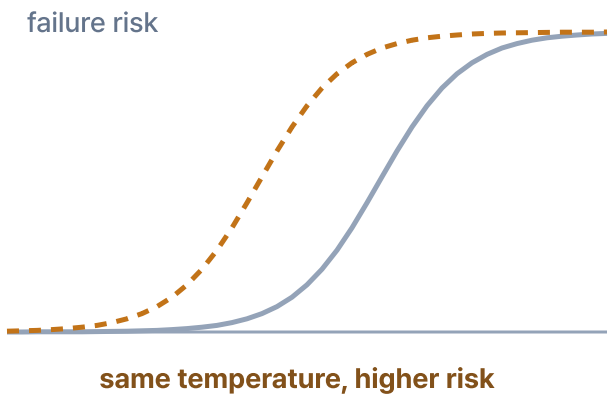

### Label shift

$P(y)$




### Concept drift

$P(y \mid x)$

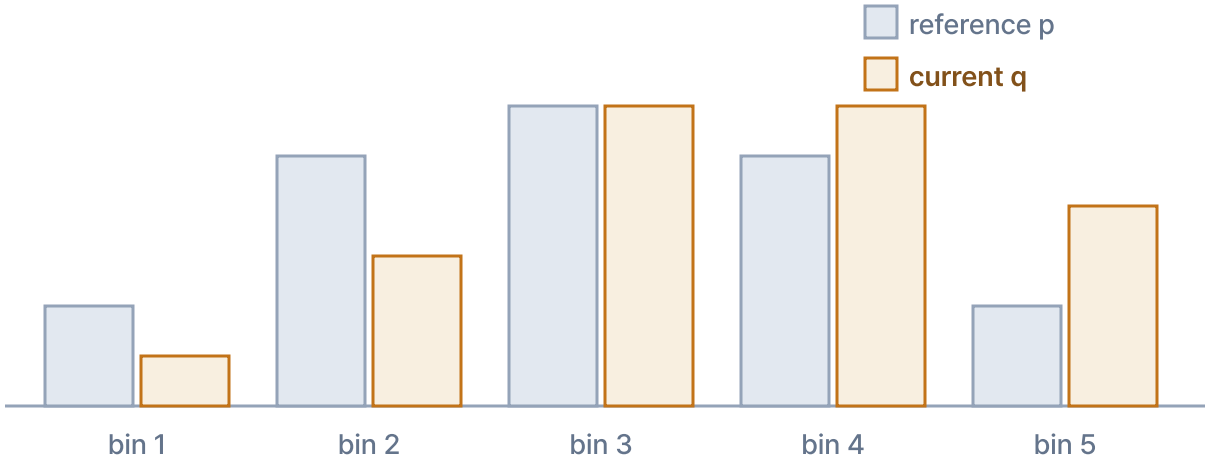



Inputs, labels, or the **relationship** can move

# Population Stability Index

PSI compares binned feature frequencies across two periods

$$PSI = \Sigma_b (q_b - p_b) \cdot \log(q_b / p_b)$$




**PSI ≈ 0.16 here**

shifts in bins 1, 2 and 5 add up

**Choose the threshold**

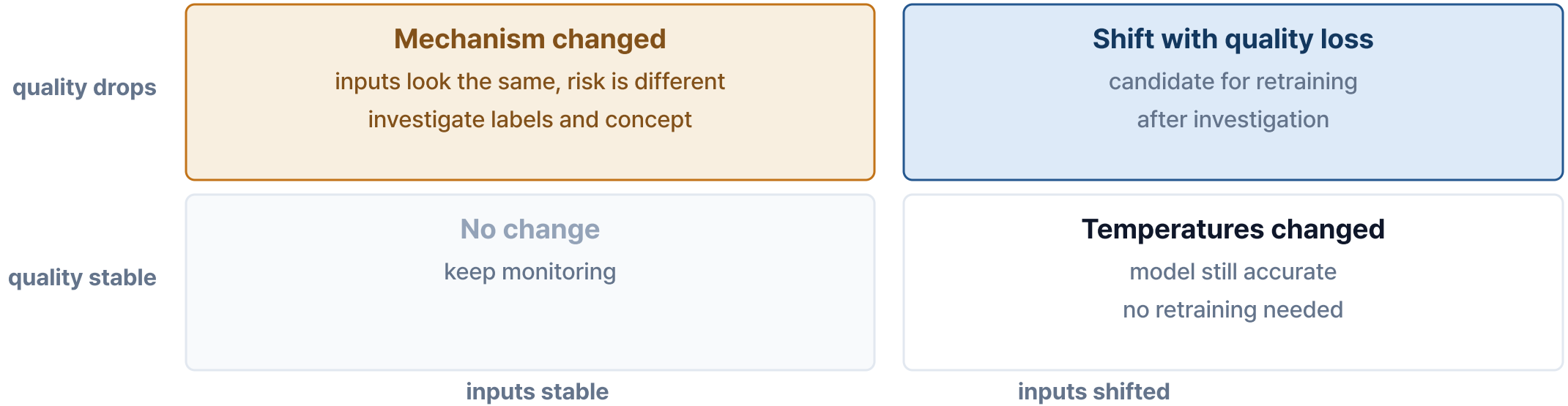
same bins in both periods

from history and the use case

PSI sums the change in each **bin**

# Caveat: Drift and Quality

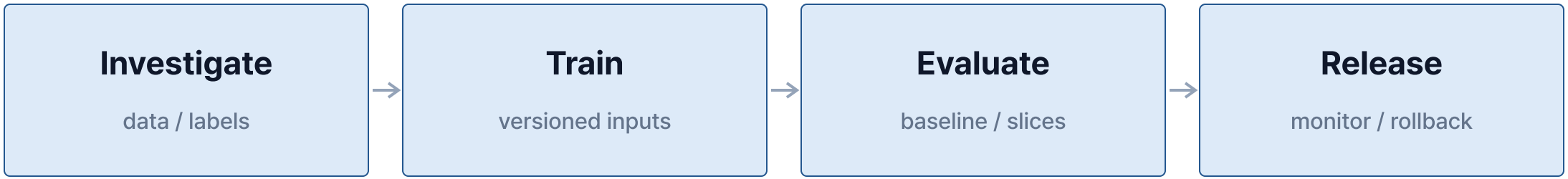
An input shift is a reason to investigate, not an automatic retraining order




Input shift and quality loss are **separate** questions

# Controlled Model Update

Validate a candidate before it replaces the serving model




Investigate → Train → Evaluate → Release

# Takeaway: Operate the data behind the model

1. Reuse the same feature definitions in training and serving
2. Version and test data, code, and models together
3. Measure drift, validate quality, and control updates